المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵎⵔⵙⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⴻⵏⵙⵉⵡⵉⵏ ⵜⵉⵜⴰⵡⵉⵏⵉⵏ
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE TÉTOUAN

GRADUATION PROJECT Report

in order to obtain the

Engineering Diploma

Software Engineering

Class of : 2021 / 2022

Implementation of a Cross User Shared Experience

Mr. Ayman BOUCHAREB

Defended on :

25th June 2022

Jury Members :

Mr ELYOUNOUSSI Yacine	Academic Supervisor
Mr TAM Borys	Professional Supervisor
Mr CHKOURI Mohamed Yassin	President
Mr MESMOUDI Yasser	Reviewer

Dedication

To My parents for their sacrifices and encouragement

To My family for their love and support

And to my friends for their encouragement and wishes of success

To all of you,

I dedicate this work.

Ayman BOUCHAREB

Thanks

First and foremost, I am extremely grateful for my parents for their love, and sacrifices, and special thanks for my father for his support and encouragement with my studies and career choices.

I would like to express my sincere gratitude to my professional supervisor Mr. Borys Tam, who guided me in the development of the project and provided me with the necessary support to complete the tasks assigned to me.

My sincere thanks go to my academic supervisor Mr. El Younoussi Yacine, who guided me through the release of this report and helped me through some hops. Also Im thankful for his patience, support, and for all the time he gave us to complete our work in the right way.

I would like to thank the staff of Nethermind for being such a welcoming team, and for being fun, good to work with and for being so helpful when needed. Special thanks to our CEO Mr. Tomasz K.Stanczak for his guidance and his constructive criticism, and Mr. Bartomiej Legid for his support during the internship and beyond, special thanks to Team Atlantis : Mr. Julius Yls, Mr. Chung Kien and Ms Anastazia Zaitceva for their encouragement, guidance, and for being fun and welcoming.

Abstract

The Metaverse since its inception has been an interesting, and a growing technology, mixing communication with interactivity introduced a new way to look at web based communication. During my graduation project I undertook the subject of developing a Proof of Concept of a Cross-Users Shared Space/experience, a metaverse if you will, that has all the basic feature minimum features.

During the implementation and contribution to the project, I took care of the development of the software, specially I took care of Networking, 3D functionalities, Avatar integration, and Server side communications and maintaining the github repo. These subjects were the highlight of my tasks, where we utilized various tools and technologies, such as Git, Unity, .Net, Browser APIs ... etc.

Keywords : Metaverse, Proof of concept, Cross-Users Shared Space

Résumé

Le métaverse depuis sa création a été un intéressant et un croissant technologie, mélant communication et interactivité a introduit une nouvelle faon de voir la communication basée sur le Web. Le sujet de mon projet de fin d'études etait le développement d'une preuve de concept d'un espace / expérience partagé entre utilisateurs, en autre mots un métaverse, qui a toutes les fonctionnalités de base minimales.

Lors de la mise en place et de la contribution au projet, je me suis occupé du développement du logiciel, notamment je me suis occupé du Networking, des fonctionnalités 3D, de l'intégration d'Avatars, et Communications côté serveur et maintenance du repertoire github. Ces sujets ont été le point culminant de mes tâches, oú nous avons utilisé divers outils et technologies, tels que Git, Unity, .Net, API de navigateur ... etc.

Mot-clefs : Metaverse, preuve de concept, expérience partagé entre utilisateurs

List of Figures

Acronyms

3D Three Dimensions

API Application Programming Interface

AR Augmented Reality

Auth Authentication

CRUD Create Read Update and Delete

ECS Entity Components System

FPS Frames Per Second

HTTP Hypertext Transfer Protocol

ID Identity

IETF Internet Engineering Task Force

IL Intermidiate Language

IPFS InterPlanetary File System

MSFT Microsoft

MSIL Microsoft Intermidiate Language

NFT Non Fungeable Tokens

P2P Peer to Peer

RPC Remote Procedure Call

RPM Ready Player Me

RTC RealTime Communication

SDK Software Development Kit

VR Virtual Reality

VS Visual Studio

W3C World Wide Web Consortium

WASM Web Assembly

WebRTC Web RealTime Communication

Contents

Introduction

The internet since its inception has been a path humans took to share data, share resources, and overall share knowledge, with the recent events that hit the world it has been proven that the internet can be used to share more than object raw data, but we can effectively share our lively-hood, with real time communication we can be with other people the other end of the pacific in milliseconds, but it the human aspect was always lacking, so the question that asks itself now, is how can we make the experience as seamless as real life? an ambitious take on web communication, so my host company Nethermind, decided to take on this problem by providing what is known as a Metaverse, this project has as a goal providing means by which users can communicate in real time while being able to interact with each other thought avatar of their choice, a gamified replica of the real world in a virtual space.
in this context our task is to develop an instance of a general purpose metaverse, and an art specific art-verse, and to realise our project we will follow the following approach :

- Chapter 1 : in this chapter we will get to know the host company, the project scope and goals, and we will know more about the team's methodology and way of tackling various aspects of the project;
- Chapter 2 : in the second chapter we will identify the requirements and needs that the system needs to fulfill, as well as the actors and target audience of said project;

- Chapter 3 : the third chapter will be dedicated to the abstract conceptualization of the project, it is a theoretical and technical study of the project do-ability;
- Chapter 4 : the fourth and final chapter will be allocated to the realisation of the project discussing tools, methods and algorithms utilised to implement various aspects of the project;

Chapter 1

General Context

This chapter presents some various aspects about the project, our work method and the host company.

The company I'll be partaking in this internship is publicly known as "Nethermind" and legally named "Demerzel Solution Limited", it is a company specialized in the development of blockchain based solutions, it tackles many aspect of the technology such as Tooling, development of the ecosystem and Infrastructure.

For the subject we will be working within the aspect of "Development of Ethereum Ecosystem", by building a Metaverse solution that tackles the problems mentioned above while building on top of Ethereum blockchain. During this internship we will be developing a Metaverse, which is a complex program of real-time systems, and networking solutions that need to be reliable, fast and efficient, and most of all scalable.

I Presentation of the Host company

I.1 The host Company

The company Demerzel Solutions LTD, better known as Nethermind, is a software company located in London, United Knigdom, its a part of the Ethereum softwares ecosystem.

Figure 1. *Nethermind Logo*

I.2 Domain of Activities

As of today the company has worked in various fields and project :

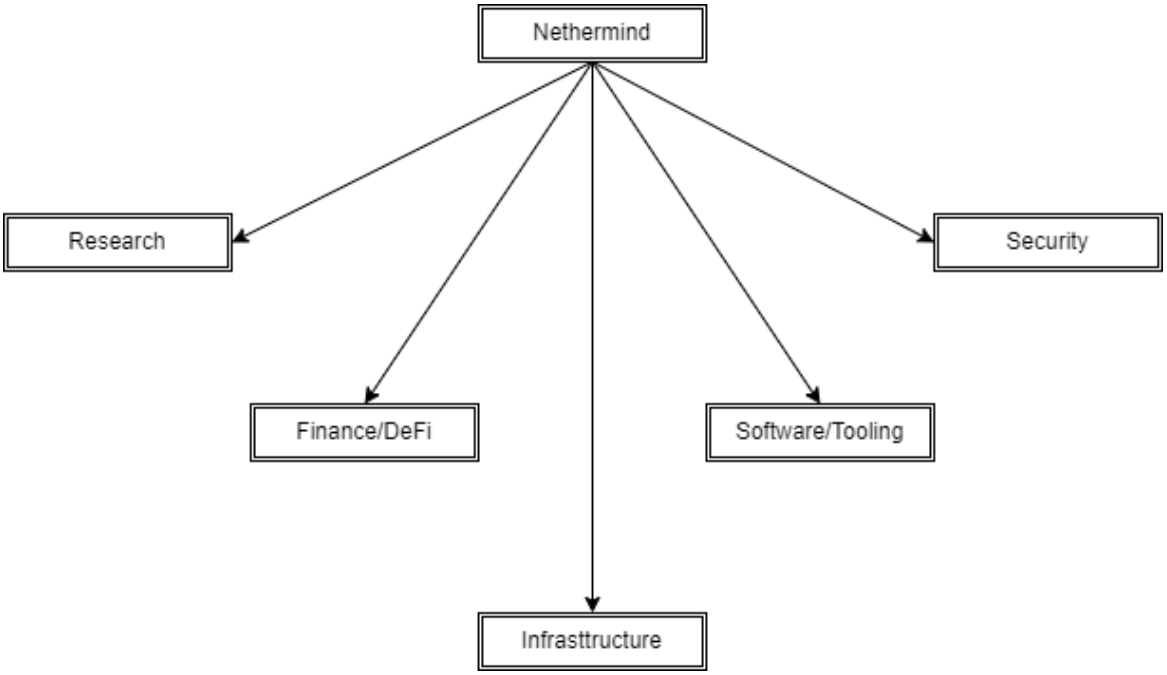


Figure 2. *Nethermind-Activities*

The companys main focus is providing the Tooling for Ethereum smart contract developers, they are responsible of maintaining one of Ethereums main execution layer clients, and they are leading research in the ways of developing and improving the experience of Crypto field in general.

I.3 Organization chart

The team Ive assigned to is called Atlantis, it is the Meta-verse research and development team, its comprised of 4 people:

- **Project Manager**

  Borys Tam.

- **Smart Contract Expert**

  Anastazia Zeitceva (Left).

- **External Relations Specialist**

  Julius Yls.

- **Interns**

  Ayman BOUCHAREB.

  Other Interns: *Parth Padawarth, Yohenba Kshetrimayum*

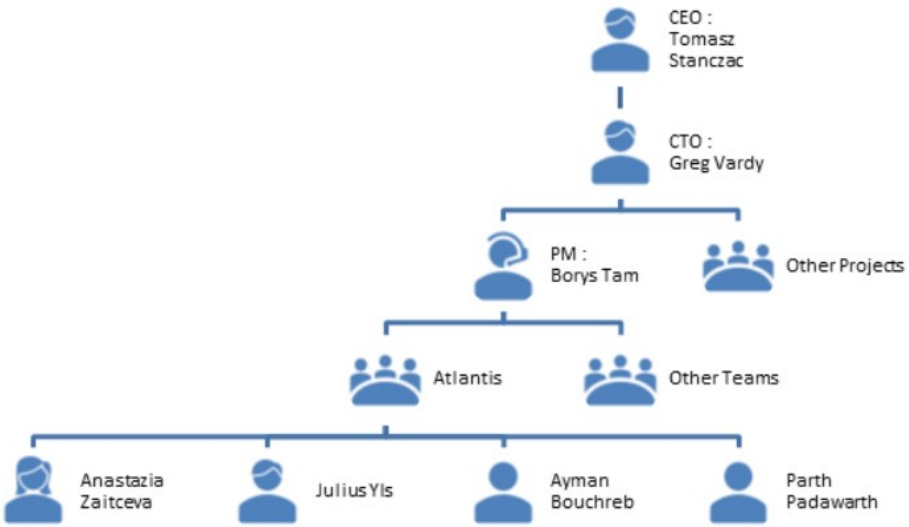


Figure 3. *Nethermind-Hierarchy*

The team Ive joined (Atlantis), is tasked by researching and conceptualizing solutions and projects in the meta-verse work field, my task inside of it was to experiment and develop the 3D space that will become known as the Meta-verse, by handling creating the various mechanics needed for a 3D gamified software, and the networking necessary for a multi-user experience inside said software, as well as suggesting solution and implementing the back-end server for the project.

II Presentation of the Project

II.1 The scope of the project

In order to provide an interactive life like experience in communication, we must first have a communication software, and a virtual space to hold these meetings, combined with the fact that Nethermind is a remote only company, we conclude that a virtual space is needed for the company to help strengthen relations between employees, as well as providing a monetisable service for external clients. My task in the project was to develop and maintain an POC of the software, and participating in problem solving and brain storming sessions.

II.2 Methodology of work

When it comes to the method followed to achieve the project, the team opted to not follow a singular methodology, arguing that introducing such restrictions and limits often introduces the mental obligation of following them, so we used what can be described as a hybrid of Agile and Waterfall, parallelising what can be done at the same time, and do in sequences what requires that, having the project as an Internal IP provided us with such liberty and freedom,

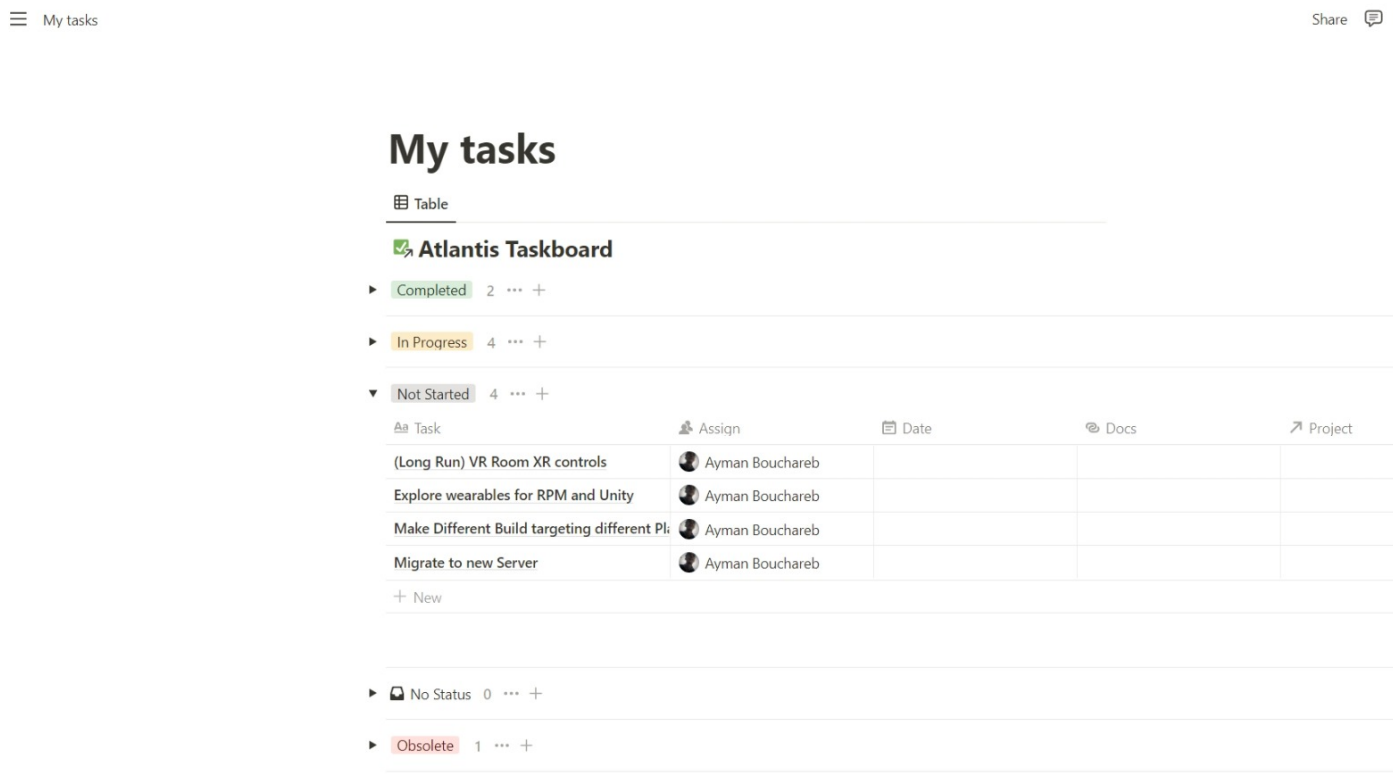


Figure 4. *Team-member-Tasks-Notion-Page*

we used the app called Notion, this application is suitable for file sharing and coordinating our team work, and within it, our project manager created a workflow emulating that of Kanban, where we have tasks created and assigned to the designated team member, and each task can be picked up and set to in progress. Every task's life cycle is as follows :

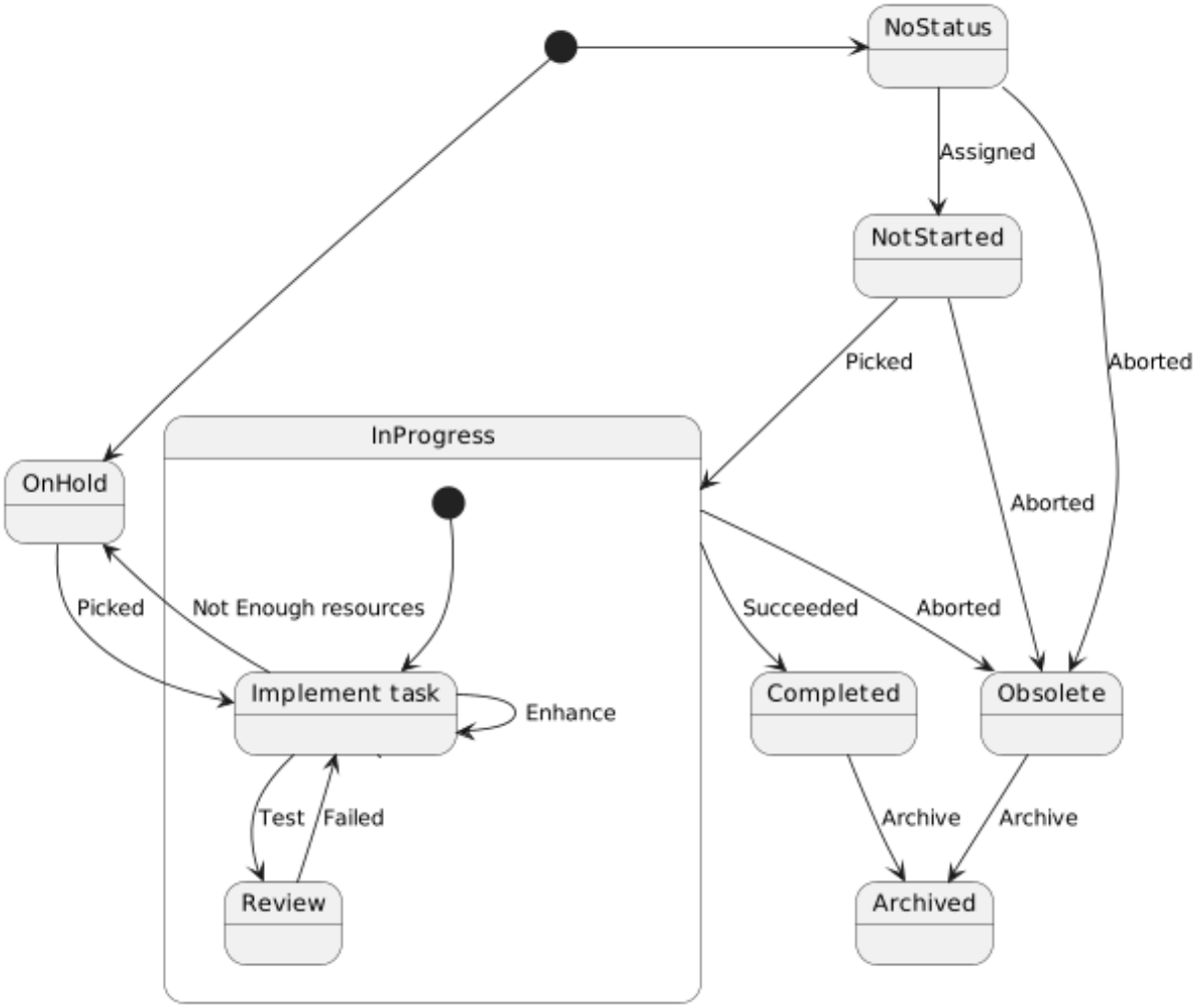


Figure 5. *Task's life cycle*

we can model the life of a task as a state machine where the states represent, well the current iteration of the task, and the actions of the team are the input, the possible states of each task are :

- **No Status**

  for tasks newly created and un-assigned to any member

- **Not Started**

  for tasks that have been assigned to a member but not worked on

- **In Progress**

  for tasks being worked on

- **In Review**

  test phase of a task

• **Archived**

tasks long aborted or finished, get moved to archived to avoid cluttering the view in the task table

• **Obsolete**

for tasks abandoned and aborted by the team

• **On Hold**

for tasks planned for the long term, non-urgent tasks, and tasks unable to be achieved

II.3 Timeline of the project

The projects timeline was not established in the traditional manner, since the project falls under a newly developed tech market, the metaverse market in itself is in the beginning of its roadmap, and since my arrival was mid development some aspects of the project were pre-established by then, but overall the expected time line was as follows :
the bellow table shows a simplified version of the time line projected for the project, the main portion of the project life cycle is dedicated toward the iterative process of developing, testing and enhancing, provided it is Nethermind's IP, the project's goal and target audience might shift during development depending on how the market is shaping and the competition.

Activity	Duration	Start Date	End Date
Training / On-boarding	3 Days	02/02/2022	05/02/2022
Specifications	1 Week	07/02/2022	11/02/2022
Design	1 Week	14/02/2022	18/02/2022
Programming	6 Months	21/02/2022	31/08/2022
Testing	6 Months	21/02/2022	31/08/2022

Figure 6. *Project timeline*

in retrospect to this time the project went according to plan, with a main demo available before the predetermined deadline, the actual time table went as follows :

Activity	Details	Duration	Start Date	End Date
Training / On-boarding	-	3 Day	02/02/2022	4/02/2022
Specifications	-	2 Week	07/02/2022	18/02/2022
Design	-	2 Week	21/02/2022	04/03/2022
Programming	Tooling Exploration	1 Week	06/03/2022	12/03/2022
Programming	Added Avatars and dynamic Object	1 Months	15/03/2022	30/03/2022
Programming	Added Networking	1 Months	01/04/2022	30/04/2022
Programming	Added Communication	1 Months	01/05/2022	30/05/2022
Programming	Added File Imports	1 Months	01/06/2022	30/06/2022
Test	The global All hands Test	1 Day	21/05/2022	21/05/2022

Figure 7. *Actual timeline*

The following figure shows the project timeline, with the main portion of the project life cycle dedicated to the iterative process of developing, testing and enhancing, provided it is Nethermind's IP, the project's goal and target audience might shift during development depending on how the market is shaping and the competition.

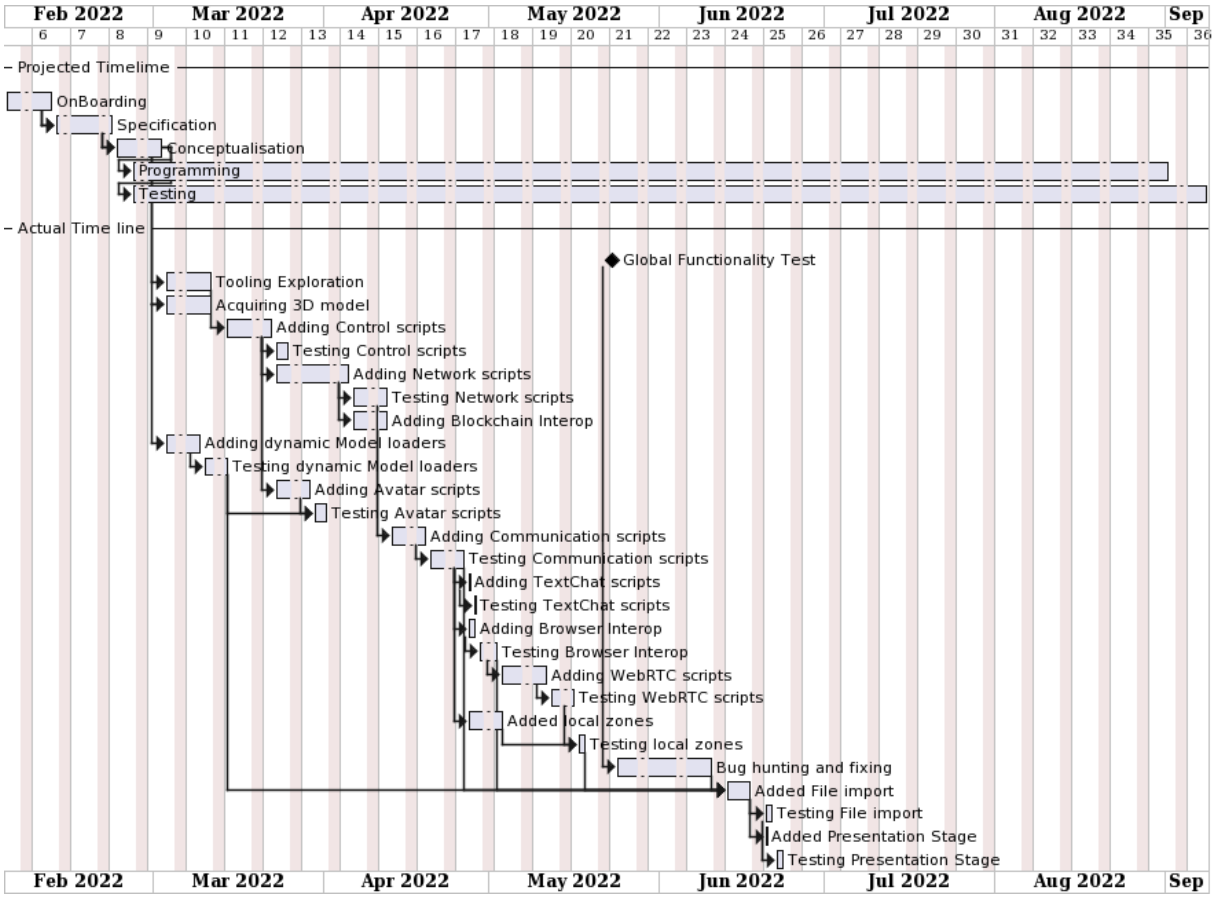


Figure 8. *the actual timeline of the project vs the projected timeline*

III Conclusion

To sum up weve discussed so far the scope of the project, which is taking place within the Nethermind company, specifically within their Metaverse team called Atlantis the project is a software web communication software with realtime 3D interactions.

Chapter 2

Specification of requirements

This chapter presents the requirements of the project, the needs that it must fulfill.

Each software in its inception must define its own requirements, and the project must be able to fulfill them. for our program we defined the main actors as being : Visitors, Hosts and Nethermind. the requirements the software must fill vary depending on the actor using it, for the Visitors, the software needs to provides means of interaction and communication, live spaces and multi-user experience, for the Hosts, the program must allow them to create, modify or delete owned spaces, it should also allow thme to have fine grained control over their spaces and monitors those spaces, and lastly for Nethermind it should allow it as an owning company to monitor and oversee the software flow.

When it comes to functional needs the software as every other software needs to be maintainable, scalable, and efficient.

I Introduction

the study of the requirements of a project constitutes an integral step of the life cycle of any project, since it helps us formalise in details what was sketched in the previous phases, and it paints a clearer idea about what has to be realised in terms of actions, actors and tasks.

II Specification of requirements

This section will server as to list the basic needs the system needs to fullfil. in order to clarify the needs of our application's users, we will identify the parties involved in it, as well as the functional requirements as well as the non-functional requirements that our project needs to obey.

II.1 Actors in the system

before we can go Identify the needs and requirements of our system, we must first identify the various actors involved in it (system). An Actor is an abstract concept modelling an external factor that may or will interact directly with our system. the project being a Meta-verse based application, we can Identify the following actors :

- `Visitors` : 3rd Party users, their functionalities are limited to communication and in app transactions;
- `Hosts` : 2nd Party users, they have full control over spaces, and virtual markets;
- `Nethermind`: Infrastructure provider, and monetary transactions facilitator;

II.2 Functional requirements

These are the functionalities of the system, they explain and specify the behaviour of each input and output of the system.

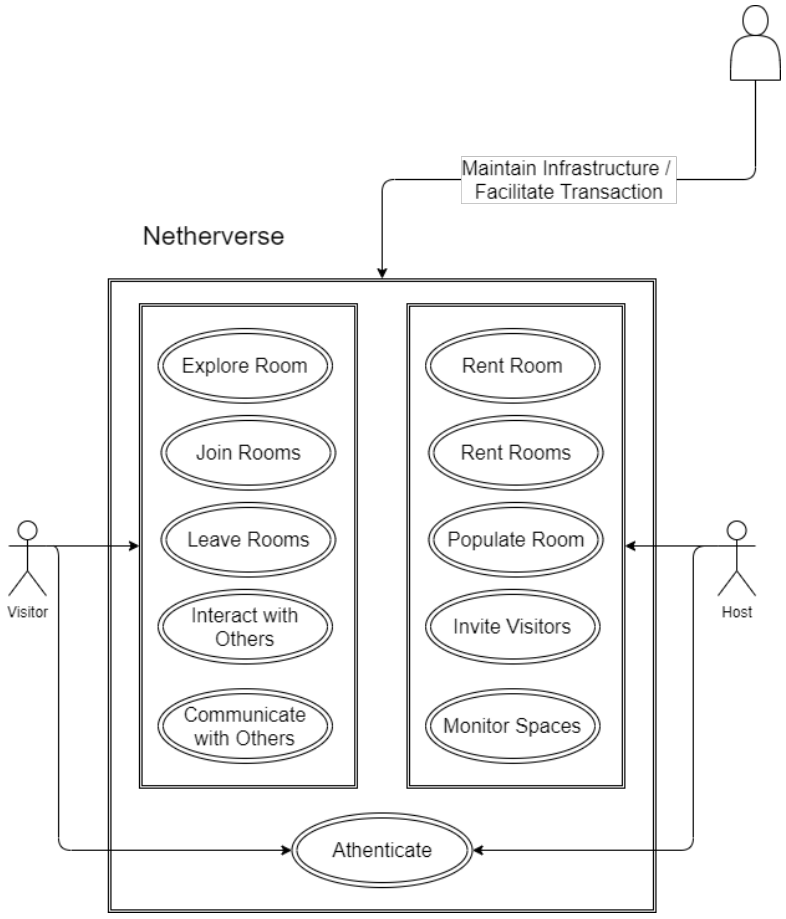


Figure 9. *Use case diagram of Functional Needs*

II.2.1 Users needs

a) Use cases of Visitors

A visitor is a non authoritative user that has limited functionalities, but they are the largest demography in our target users, the functionalities associated with them are :

- Join Immersive VR/AR/3D spaces;
  - Move inside the space;
  - Leave the Space;
  - Alter the Space;
  - Interact with other Users;
  - Interact with Objects / Arts;

- Explore Existing 3D Spaces;
- Create Temporary 3D Spaces;

b) Use cases of Hosts

A host is a user who can create persistent 3D spaces and populate them with Miscellaneous Objects but most importantly they can have spaces that allow monetary transaction within them, their main needs are :

- Create Immersive Virtual Reality (VR)/ Augmented Reality (AR)/ Three Dimensions (3D) meetings;
  - Build a shared space;
  - Import models;
  - Share the space;
  - Delete the space;
  - Define transaction rules;
  - Interlink owned spaces;
- Populate Spaces with Arts / Objects;
- Invite Visitors (in case the space is private);
- Monitor Spaces and collect metadata;

II.2.2 Textual Description Of use cases

In this Section we will represent some of the use cases of the system, and their functionalities, in the form of tables that show various metadata about each use case, such as the Actor, the pre-condtions needed for such action to be possible, the post-conditions that will be achieved after the action, and the expected output, as well as a simple overview of the process itself.

a) User registration

Use Case	Register
Description	The Actor Registers in the system
Actors	• Host • Visitor
Start Event	Visiting the System (Website, Apps)
Basic Scenario	• the Actor visits the webiste/Apps • the actor enter their credentials • the system verifies the data
Variations	The actor is potential : Visitor — Host
Pre-Conditions	Actor un-registered
Post-Conditions	Actor registered
Constraints	-

The following is a overview diagram of User registration process :

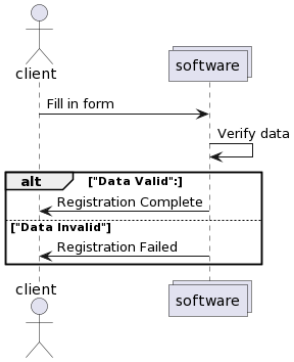


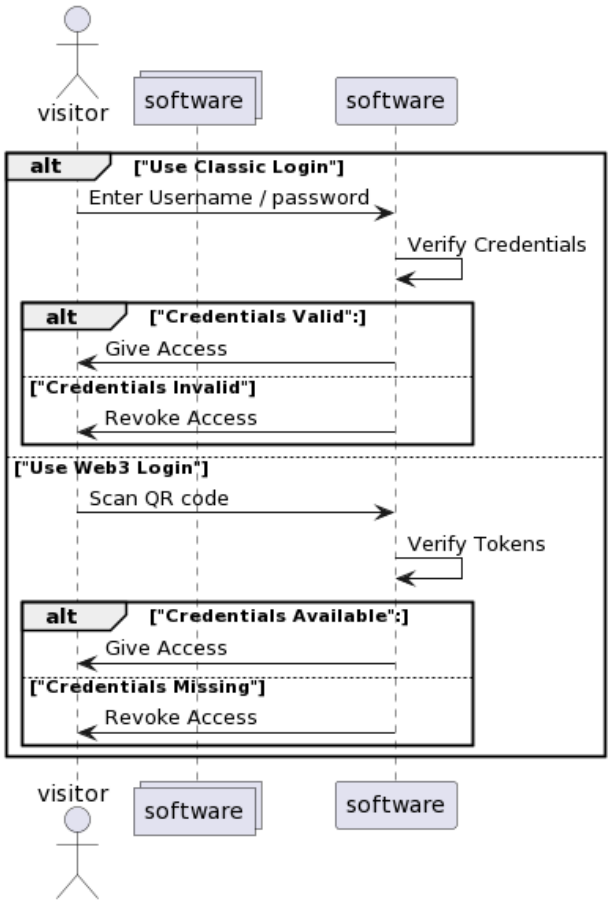
Figure 10. *Sequence diagram of User registration process*

The figure shows the registration process, which can be simplified in the following simple steps : the actor registers in the system, and the system verifies the data, and then the actor is registered.

b) User Authentication

Use Case	Authenticate
Description	The Actor Authenticate with the System
Actors	• Host • Visitor
Start Event	Visiting the System (Website, Apps)
Basic Scenario	• the Actor visits the webiste/Apps • the actor enter their credentials • the system verifies the data
Variations	• the Actor authenticates using Classic logins : Email, Password • the Actor authenticates using WEB3 tokens : wallet + User token
Pre-Conditions	Actor un-authenticated
Post-Conditions	Actor has access to the build
Constraints	Actor must have been registered in the system

The following is a overview diagram of User authentication process :



caption[Sequence diagram of User Authenticatio]Sequence diagram of User Authenticatio

The diagram shows the simple steps a user can take to register, it shows that the user can either, chose to login in the old fashioned way using email and password, or using web3 authentication with a wallet and a user Id token.

c) User logout

Use Case	Logout
Description	The Actor logs out of the System
Actors	• Host • Visitor
Start Event	Clicking logout or exit
Basic Scenario	• the Actor clicks on Exit button • the app navigates to the main page
Variations	• Actor Exits The app • Actor clicks Logout
Pre-Conditions	Actor authenticated
Post-Conditions	Actor un-authenticated and has no access to the spaces
Constraints	Actor must be authenticated

The following is a overview diagram of User logout process :

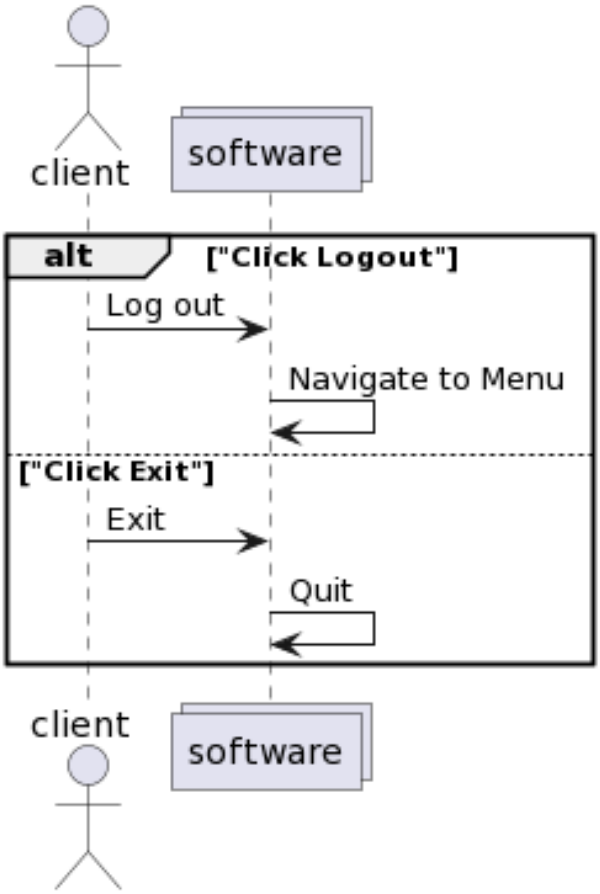


Figure 11. *Sequence diagram of User logout process*

As with most applications logging out can happen either by explicitly clicking on the logout button, or by closing the app, which is exactly what the diagram above showcases.

d) User join room

Use Case	Join Room
Description	The Actor Joins a Virtual Space in the app
Actors	Visitor
Start Event	Selecting a space
Basic Scenario	• the actor browses the spaces • the actor joins that space
Variations	• The actor joins his public spaces • the actor joins private spaces
Pre-Conditions	3D virtual world unaccessible
Post-Conditions	player spawns in a virtual space
Constraints	Actor must be authenticated

The following is a overview diagram of User joining spaces process :

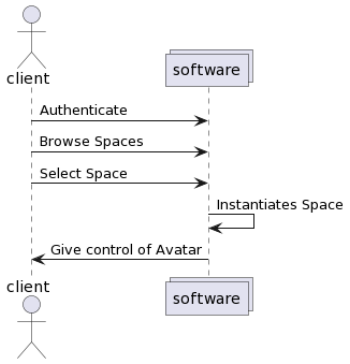


Figure 12. *Sequence diagram of User join room process*

the diagram shows the steps a user can take to join a space, it shows that after authenticating, the user is presented with a list of spaces, and can select one to join, once joined the system handles instantiations of necessary artifacts, and then it gives back control to the user.

e) User Room exploration

Use Case	Explore Room
Description	The Actor Explore Virtual Spaces in the app
Actors	Visitor
Start Event	Joining The Space
Basic Scenario	• the actor joins a space • the actor gets the avatar • the controls get activate • the actor can move the avatar
Variations	• the actor can use touch controls • the actor can use VR controls • the actor can use Keyboard/Mouse controls
Pre-Conditions	Avatar un-accessible
Post-Conditions	avatar present and controllable
Constraints	Actor must join a space

The following is a overview diagram of User exploring process :

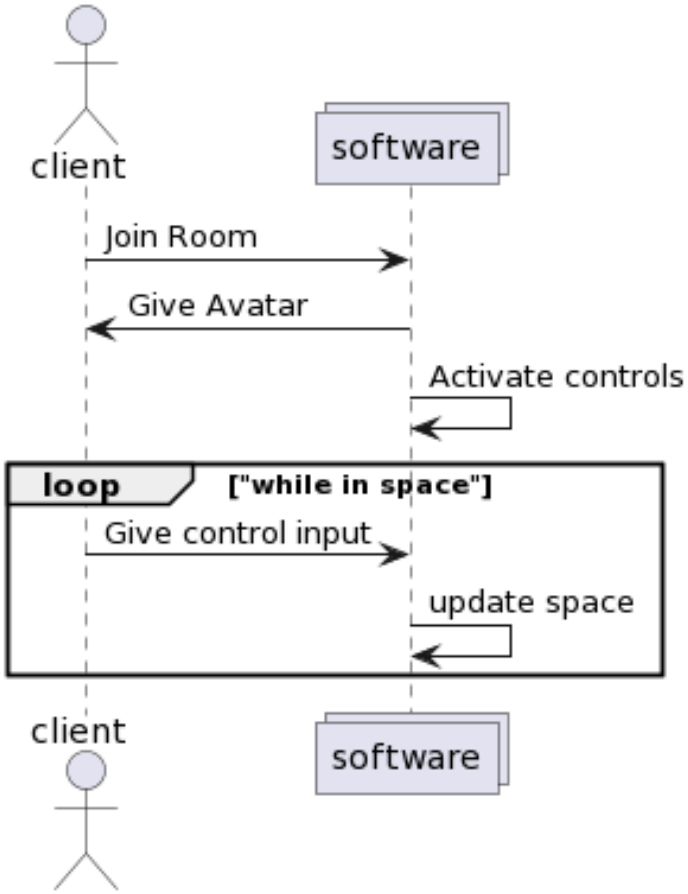


Figure 13. *Sequence diagram of User Room exploration*

After joining a space, the User gets control of his avatar, and can move it around in the space.

f) User leave room

Use Case	Leave Room
Description	The Actor leave the virtual Space
Actors	Visitor
Start Event	Clicking on Leave Space
Basic Scenario	• the actor click on Exit button • the system navigate to main menu
Variations	• Actor Exits The app • Actor Leave the space
Pre-Conditions	Actor state in Space
Post-Conditions	Actor state out of space
Constraints	Actor must be in a space

The following is a overview diagram of User leaving process :

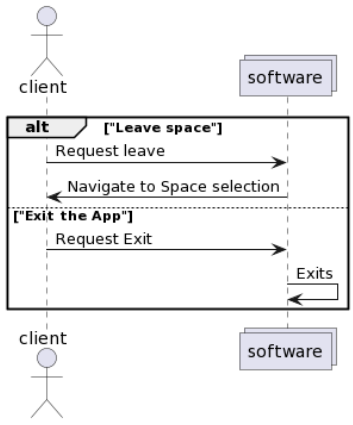


Figure 14. *Sequence diagram of User leave room process*

While in space the user can leave, either via the logout button, or by simply exiting the application.

g) User in space interactions with users

Use Case	Interact With Users
Description	Actor can communicate and Interact with other users in the shared space
Actors	Visitor
Start Event	Joining The Space
Basic Scenario	• the actor joins a space • the actor gets In space UI • the controls get activated • the actor can use UI to communicate
Variations	• The Actor Uses Voice chat • The Actor Uses Text chat • The Actor Uses Emotes
Pre-Conditions	Actor cant interact with other users
Post-Conditions	Actor Interacts with users in the shared space
Constraints	Actor must be in a space

The following is a overview diagram of User / User Interactions (communication) :

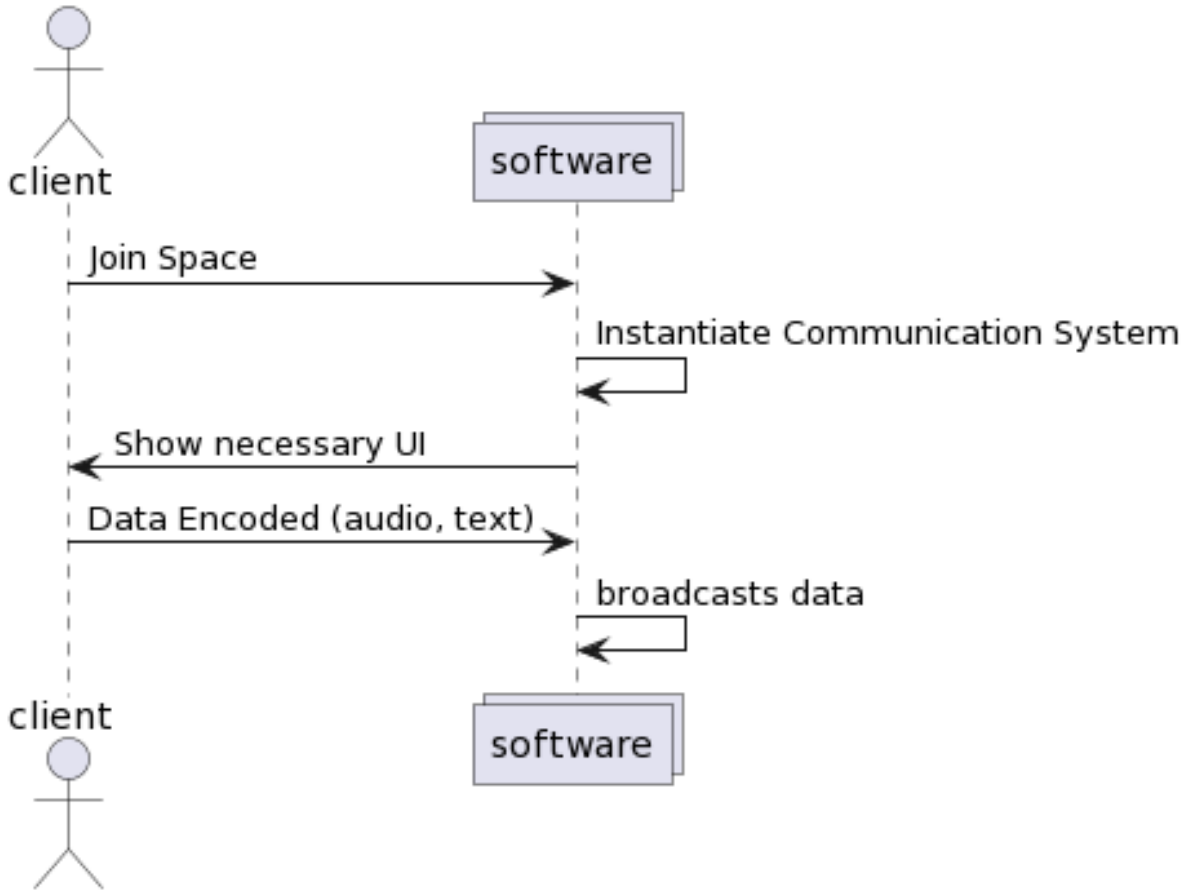


Figure 15. *Sequence diagram of User in space interactions with users*

The diagram above shows the steps taken to handle user to user communication, so first the user joins a space, then the system bootstraps some necessary components for communications, and then it starts capturing data from the user in order to broacast it to the other users, that data is mainly encoded text or audio buffers.

h) User in space interactions with inanimate objects

Use Case	Interact With Objects
Description	Actor can communicate and Interact with other users in the shared space
Actors	Visitor
Start Event	Joining The Space
Basic Scenario	• the actor joins a space • the actor gets In-space UI • the controls get activated • the actor can use UI to alter in space objects' state
Variations	• The Actor scales the object • The Actor rotates the objects • The Actor moves the objects • The Actor adds new objects • The Actor removes existing objects
Pre-Conditions	Actor cant interact with other objects
Post-Conditions	Actor Interacts with objects in the shared space
Constraints	Actor must be in a space

The following is a overview diagram of User / Object Interactions :

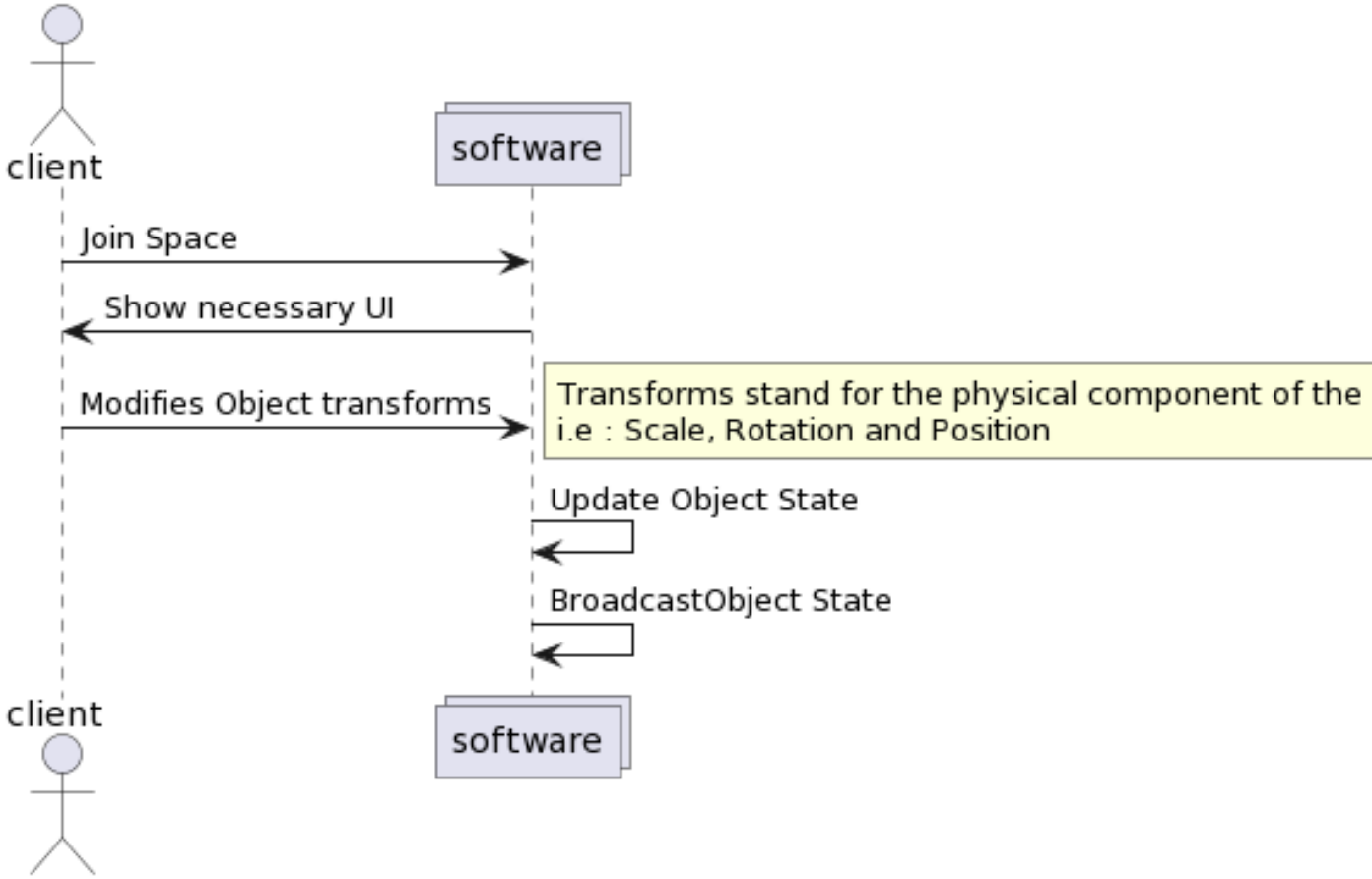


Figure 16. *Sequence diagram of User in space interactions with inanimate objects*

Basically once inside a space, the user can alter objects marked as modifiable, and the system will broadcast the changes to the other users, the user can modify their transform component and these changes will be reflected on other machines as well. User can Also add new objects or remove objects from scenes.

III Specification of non-functional requirements

After having determined the functional needs, we present below all the constraints to be respected to guarantee the performance of the system while respecting the requirements of the user.

- Performance : Our application must ensure a minimum response time while meeting the needs of the manipulator;
- Simplicity : Every user will be able to use this application in an easy and clear way;
- Ergonomics : Interfaces should be simple and user-friendly;
- Modularity : Have simple code that is easy to maintain and understand when needed;
- Maintainability : the project's inner working should be maintainable and conform to industry standards;
- portability : the project needs to not be tied to platform specific technologies to allow for build target portability;

IV Conclusion

This chapter adressed the various requirements the projects needs to follow and fulfil, mainly from the visitor's and the host's points of view, which can be summarized in "democratized metaverse building", this chapter also defined some basic non-functional needs the project has, such as modularity, portability, and Maintainability.

Chapter 3

Design

This chapter documents the design of the system, and the technical decisions made.

Every software goes through a phase of design, and the design process is one of the most important part of the software development process. since it give us a clear picture of the system, and the parts involved in it. For our program we made the design in two phases, an Overview and a Detailed Design.

In the overview we discuss the logical architecture being user i.e : Entity Component Systems Architecture, and the Physical architecture being the hardware.

Then in the in depth phase we discuss the various sub-systems involved in delivering the functionalities of the system. and how they interact with each other, and how they are connected to the main system.

I Introduction

In this chapter we present the various concepts developed for the project, in both manners : general and detailed.

II Preliminary Design

In this part we approach the definition of the technical architecture which consists in making the choice of technologies and organization of software components best suited to the needs and constraints of the host organization. These choices are then relayed within our project, guiding the design and allowing the transformation of a functional model into an efficient and robust application.

II.1 Logical Architecture

In this section we will describe the logical architecture of the project, and how its conceptualized systems interact and behave theorically.

a) Entity Components System (ECS)

Since the project's nature is unconventional by nature, having it resembles a game more than a classical software, we opted to use a Model refined for such purposes thus why we chose ECS, i.e : Entity Components System, ECS's architecure follows the principles of Composition over Inheritance, it softly rejects the OOP in favour of a more adapted and generic System, in ECS entities are not defined as being part of a complex hierarchy, but instead they are defined as being part of a sub-system, these components and sub-systems form a global system that acts globally over all entities given they fullfil the contracted interface (i.e : they have the required components), the basic parts of ECS are Components, Systems and Entities :

- Entity : this is typically a general purpose Object, i.e : data plus behaviour;

- Component : this is an entity that possesses a particular aspect, and holds data needed to model that aspect;
- System : A system is a process which acts on all entities with the desired components;

In an analogy we can loosely compare ECS to Interfaces in OOP and functions, as in a function who's type signature takes an interface, requires the objects it operates on to fullfil the contract implied by said interface, so in this analogy a system is model by the function while the entity is object parameter and the entity is the interface implementation, while interfaces can't have default implimentations, Components do have bodies.

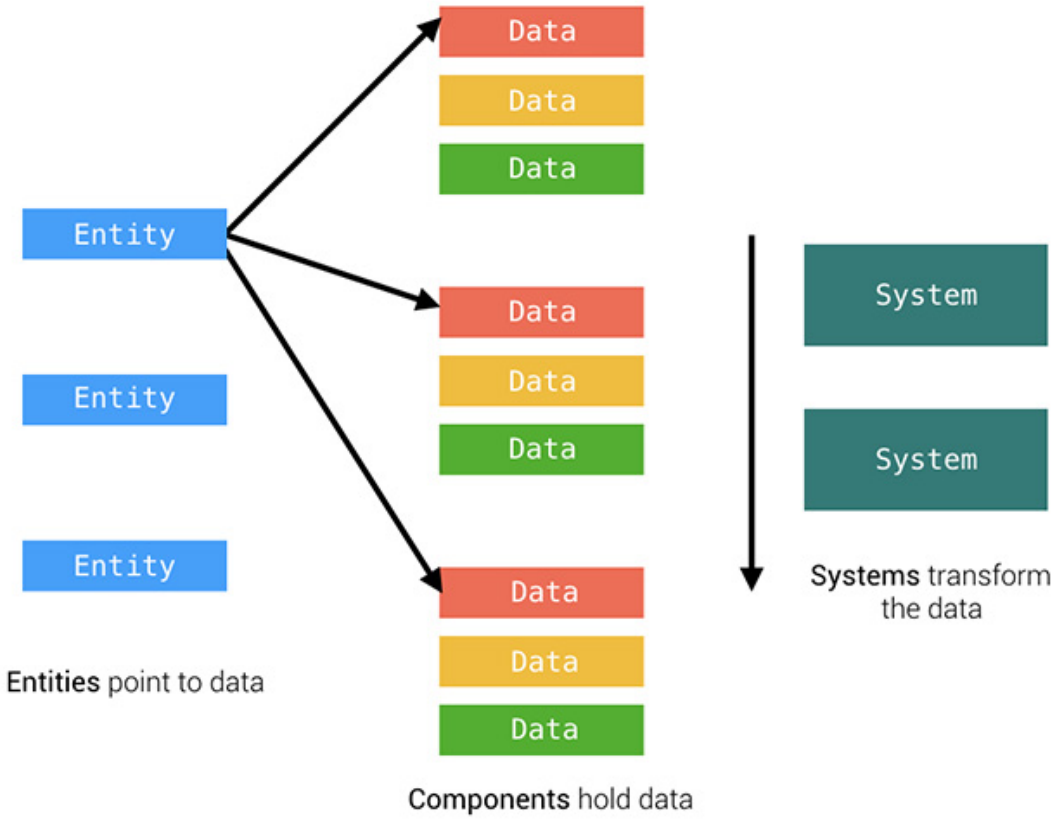


Figure 17. *a chart of the basic components of ECS*

II.2 Physical Architecture

The architecture of our application has three main constituents, the client, the networking server and the signaling server, these parts co-operate to give the product its overall behaviours. The

system uses a mixture of P2P technologies, and centralized solutions to achieve its goal. hence the client itself serves as a server node for some functionalities.

- Client : consists of the metaverse's virtual space, alongside the various interfaces developed to enrich and ensure a swift use of the software, it also serves as a communication Peer;
- Networking Server : this is the server tasked by maintaining and controlling the shared state across multiple users;
- Signaling Server : this is the server that spins media-calls and media sharing within the spaces, it takes care of bootstrapping Peer to Peer connections;

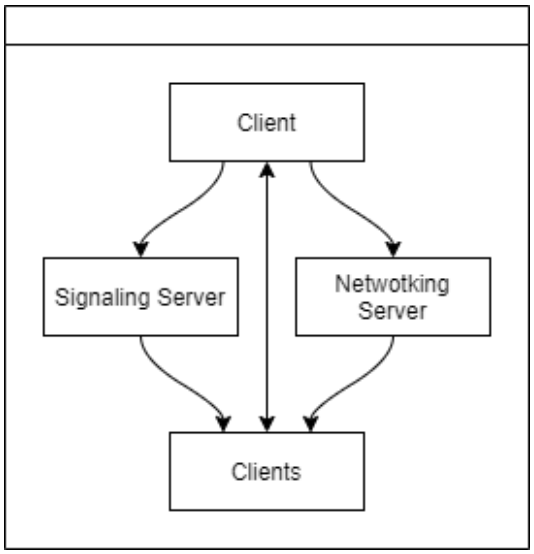


Figure 18. *the simplified Software architecture*

III Detailed Design

The project process is a interaction between various internal and external entities, but the flow of action is as generally as follows :

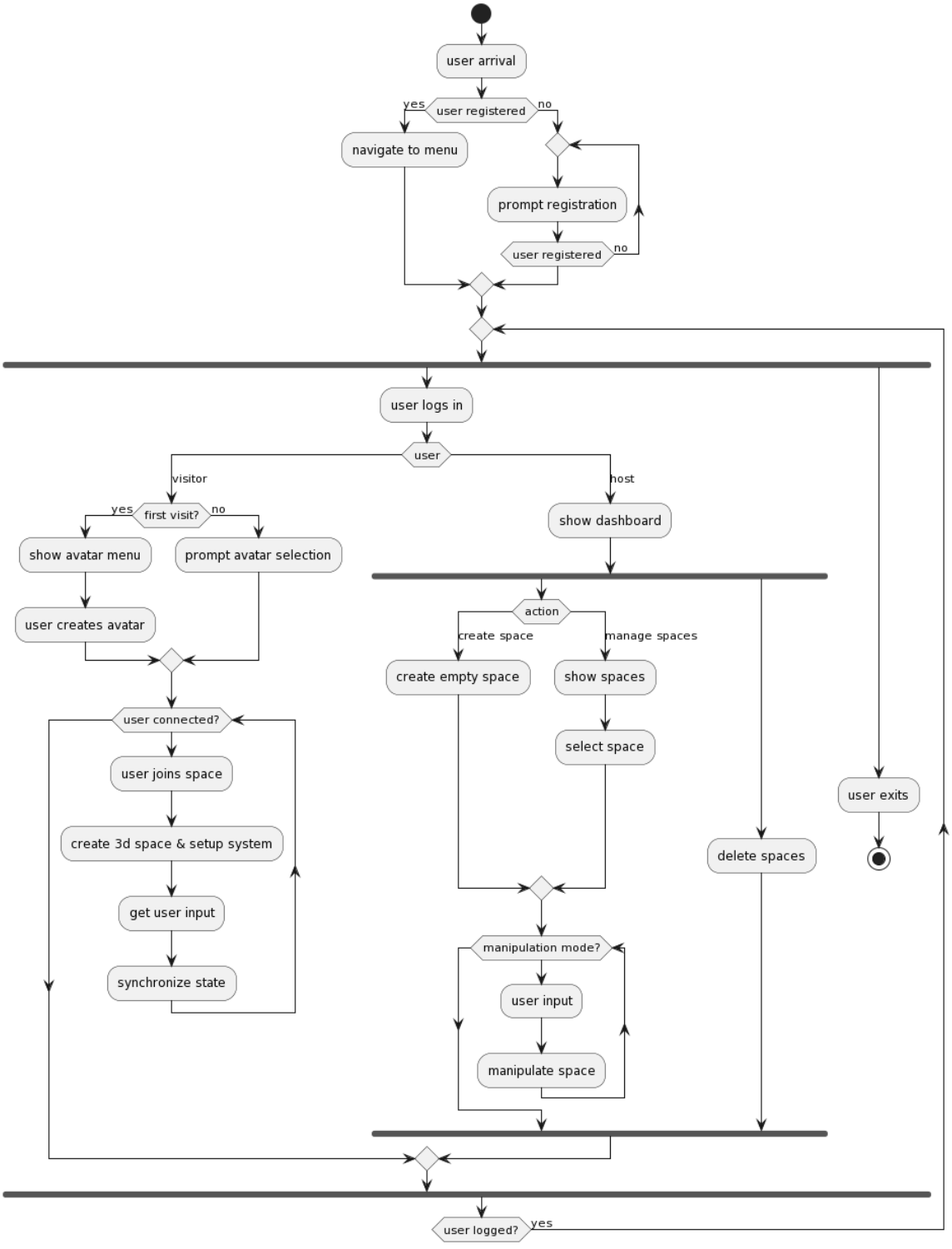


Figure 19. *the general flow of actions in the program*

to give more context to the diagram above, we will delve into a more detailed Conception in the next paragraph.

But first let's paint the global image, the system's core processes and concepts are :

- Accommodation of users;
- Giving ability to create / expand spaces;
- Giving access to shared spaces;
- Ensuring having a shared cross users Experience;

Starting by ”Accommodation of User”, this process relies on the User either registering or Logging in the system, the user can be verified either by OAuth, Classical Login System, or by blockchain/web3 Identity (ID) tokens, a user is allowed to log in if and only if they have a space or they are trying to log into a public space, or they have been invited into a private place (an invite is can be in the form of an Non Fungeable Tokens (NFT)).
Then there is ”Giving access to shared spaces” which is the process of allowing Users to see and explore available spaces with each other (given they are allowed in those spaces), those spaces are then part of the Metaverse.
Next comes ”Giving ability to create / expand / extinguish spaces” which is summarizes the process of allowing people to create their own spaces as they see fit, decorate them and expand on them to build communities, these functionalities are what allows this Metaverse to be ”Dynamic”.
and Finally ”Ensuring having a shared cross users Experience” which is the requirement of having one shared state and one shared experience across users no matter what, this describes the set of functionalities and systems that allows for multi-users interactions, communication inside spaces, and it is what makes the Metaverse ”Alive”.

III.1 Client architecture

The client is the public interface that is accessed by the visitors, it has multiple subsystems layered on top of each other to achieve various taskes, first we have the Unity webgl build, that takes care of rendering the 3D spaces, then we have the communication system, the P2P video/voice/text chat mechanism used by the system, and lastly there is the interop layer, that

consists of a set of type definitions and function implementations that allow both subsystems to pass data back and forth.

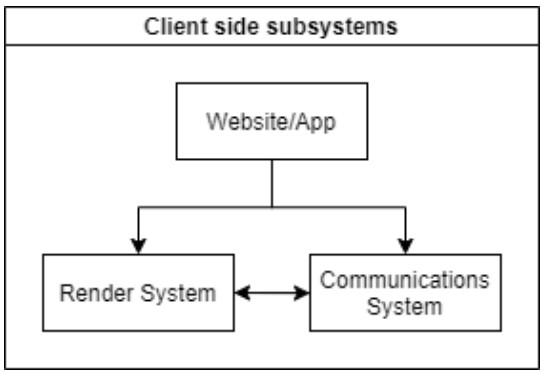


Figure 20. *The client side simplified architecture*

a) The Virtual Space

This is the Engine that takes care of the 3D rendering, in the true fashion of ECS, the system model is comprised of many subsystems each taking care of an aspect of the program.
The subsystems can be categorized in two categories, worker subsystems, and actor subsystems.

- Worker Systems : Worker systems are those that take care of background functionalities, they start as soon as the session starts, and they function independently of user Input, their functionalities vary from bootstrapping other subsystems, to delivering message between them;
- Actor Systems : Actor systems are the subsystem that awaits user input in order to perform specific action depending on that action, they vary from the chat subsystem to the file Importer system;

the map of these systems is intertwined since they depend on each other, and together they form the Virtual Space system.

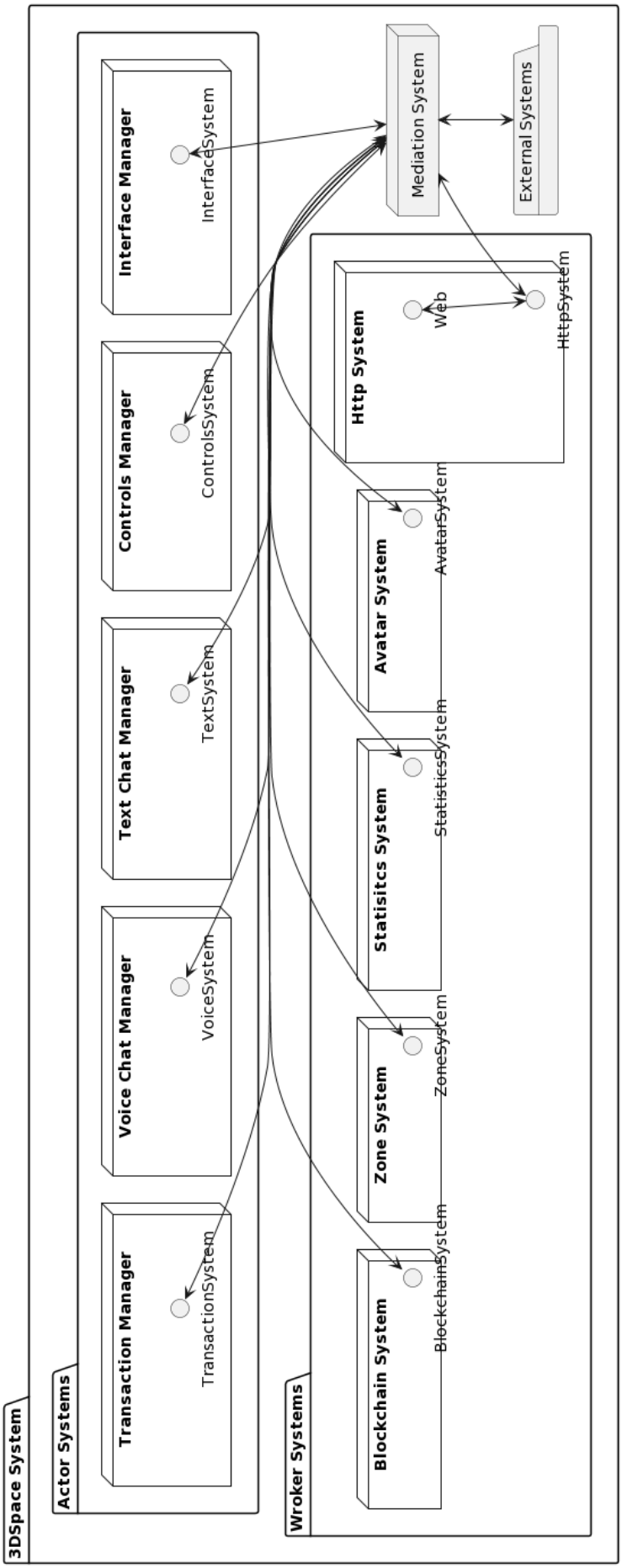


Figure 21. *this is an extremely simplified view of the system*

the figure above shows the various system working in conjunction to each other, in details they function like the following.

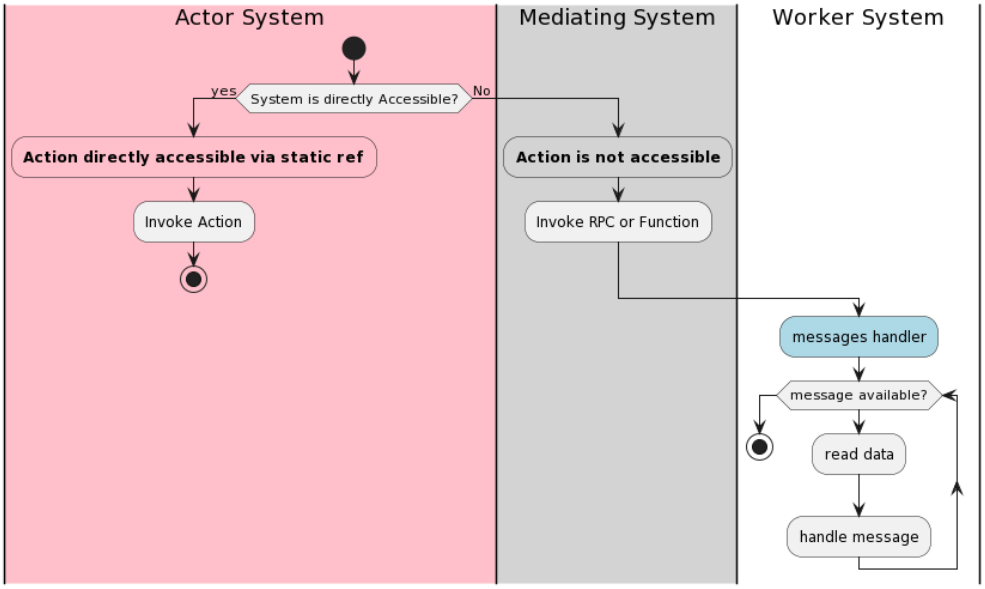


Figure 22. *this is an overview of inter-system interaction*

Actors Systems' entry point is user interactions, those systems will try to handle those events internally if possible, either via invoking the event handler, or by acquiring a static singleton ref of another system, if neither exist then that means the other system is either remote, external or a standalone entity, all of those are accessible via the mediation manager, the invoking system then bundles a message that include metadata about the action required, input arguments to pass to the handling system and Transfer Pipe, and passes that message to the Mediation Manager then the latter transfers the message to the desired system. As an example of these interactions we will look at how the system as a whole handles the action of players moving from zone to zone (The figure ignores the case of Stage::Zones, or Store::Zones for the sake of simplicity).

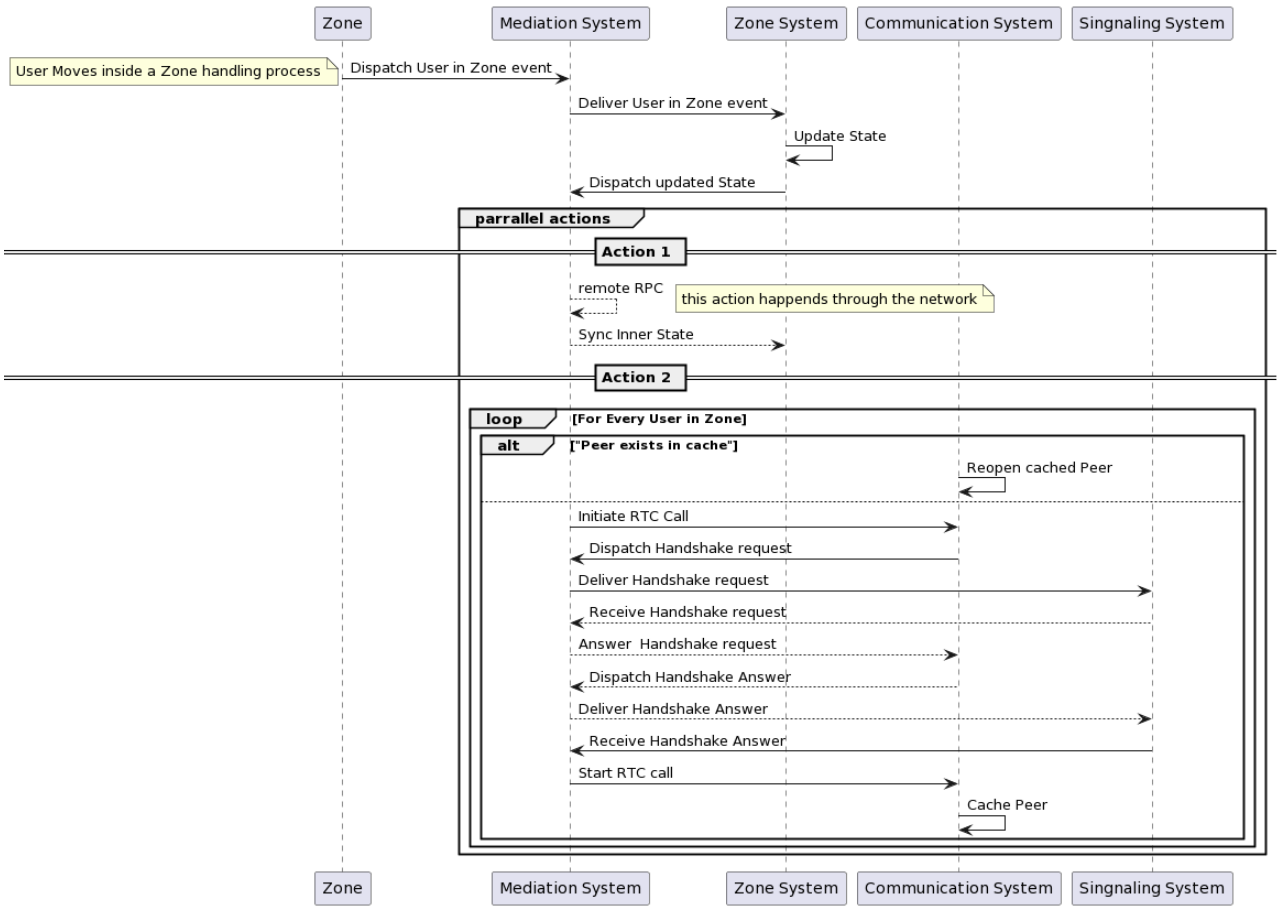


Figure 23. *User joined zone event handling process (dotted lines express remote processes)*

The figure shows the different interactions between the systems to initiate an RealTime Communication (RTC) call, another example would be the File Import system, which works as follows :

The user uses the file explorer to select a file, then the file import system uploads the byte array of that file to the cloud, and awaits for a response wrapping a URL, then it broadcasts that URL to all users in that space, and using the Metadata bundled in the broadcasted message, the system creates the necessary objects to manifest the media file in 3D. The following figure shows the process as it happens (it ignores the presentation type of files for the sake of simplicity):

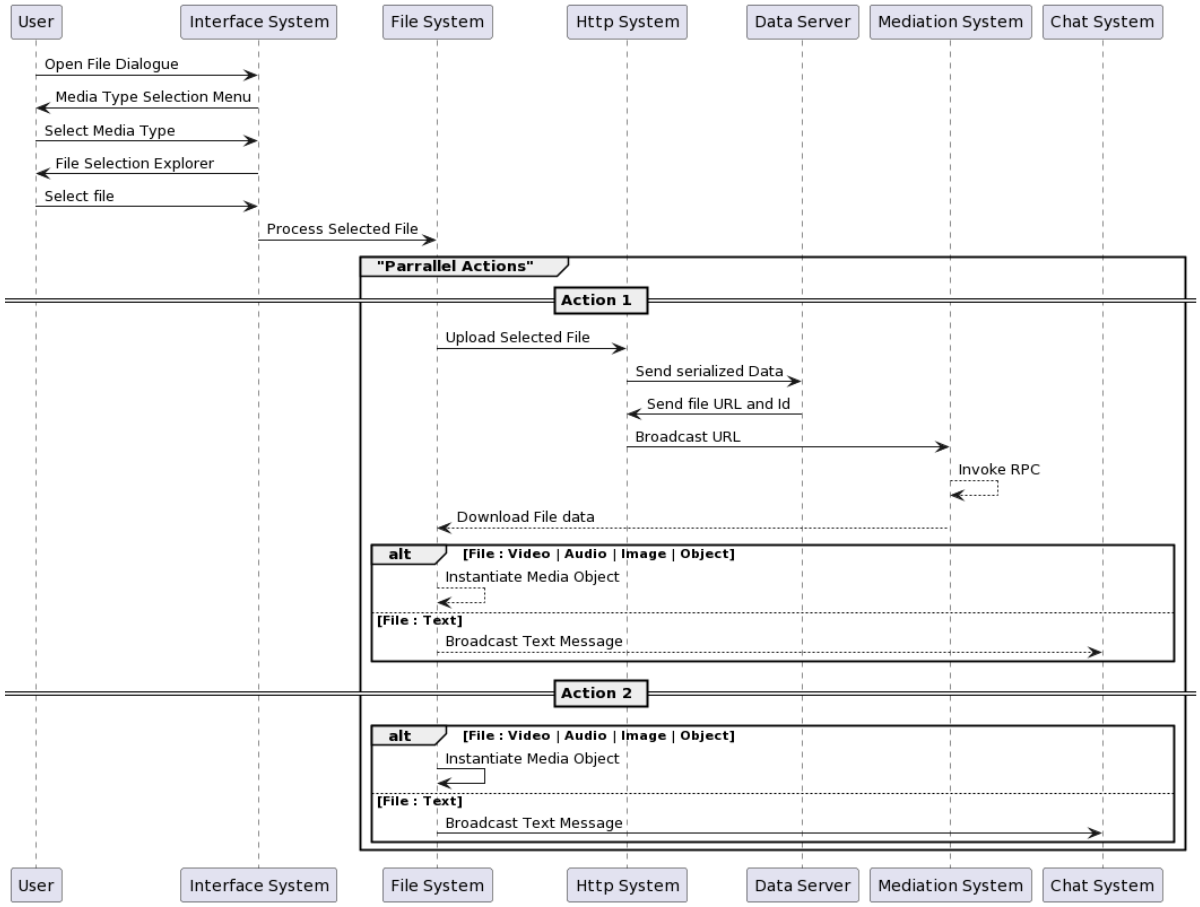


Figure 24. *File import sequence diagram (dotted lines express remote processes)*

b) The Communication Layer

The communication layer relies on the Web RealTime Communication (WebRTC) technology, it is developed as part of the host environment instead of being part of the inner managers' ecosystem.

Web real time communication WebRTC is a public and open source Application Programming Interface (API) and set of Protocols that aims to provide functionalities facilitating creating rich media based communication, it is published by the World Wide Web Consortium (W3C) and Internet Engineering Task Force (IETF), and supported by all major browser vendors such as : Microsoft, Apple, Mozilla and Google.

One of the main aspects of WebRTC is that it only creates direct connections between 2 client devices, which poses some problems regarding scalability.

Spinning a peer to peer connection requires the existence of an external factor that would help broadcast the credentials of the callers to the callee and vice versa, this entity would be the signaling system.

The system works as follows :

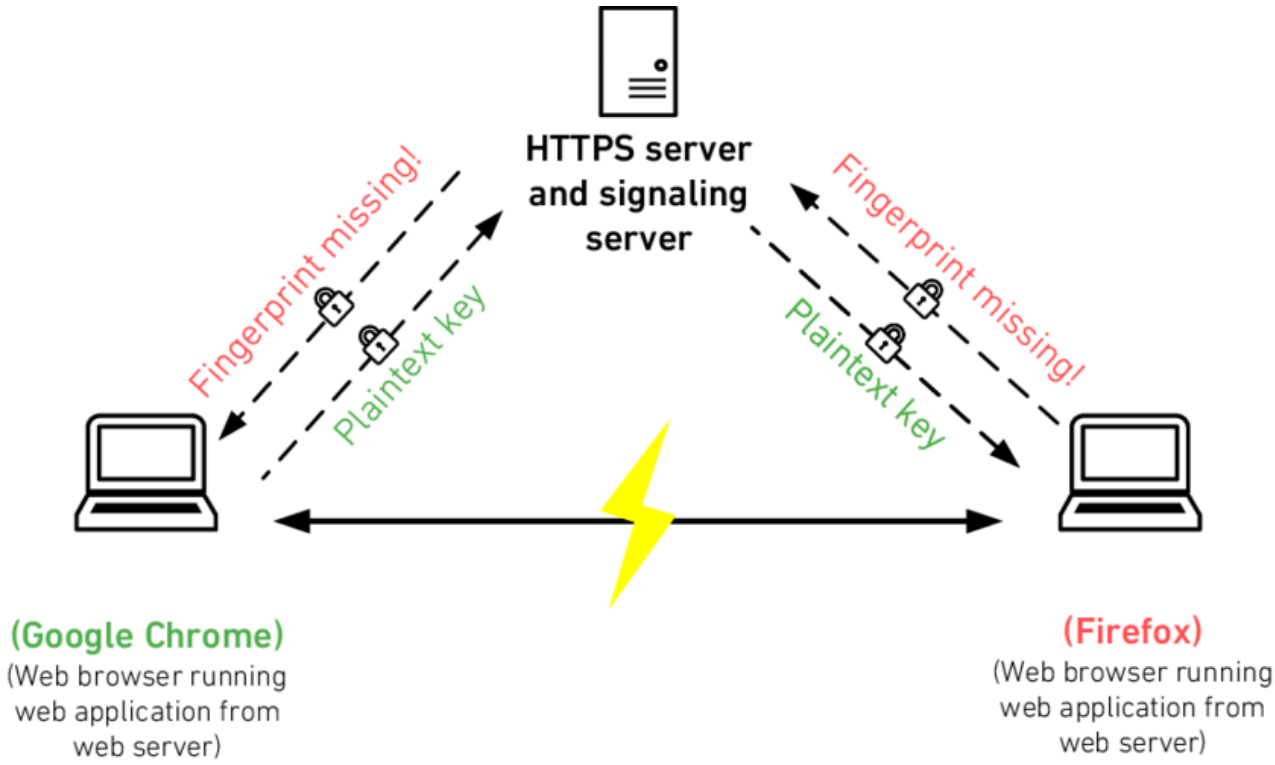


Figure 25. *The overview of a WebRTC call initiation*

This Peer to Peer (P2P) architecture poses difficulties in maintaining it, but provides a great flexibility when dealing with it [**1**].

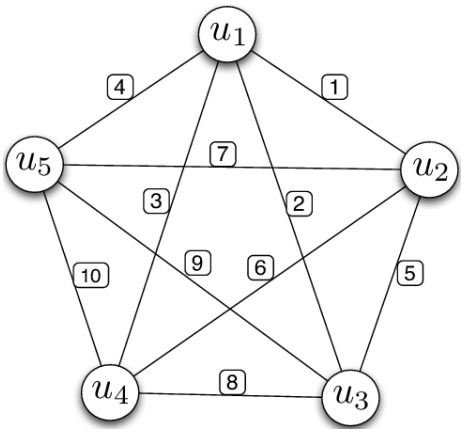


Figure 26. *the P2P connections form a complete graph*

i. Communication Layer implementation The communication layer is the subsystem that takes care of starting, maintaining and ending WebRTC calls, it does so by initiating what we call a ”Handshake”, the Handshake is a process of a sequence of messages that goes back and forth between the Peers, and these messages include some metadata about the host devices that would be crucial for the RTC call to start. The Protocol has been designed to be transfer agnostic, i.e it does not specify any technology for the broadcast of these messages, but certainly one cannot start an RTC call without the process of signaling.

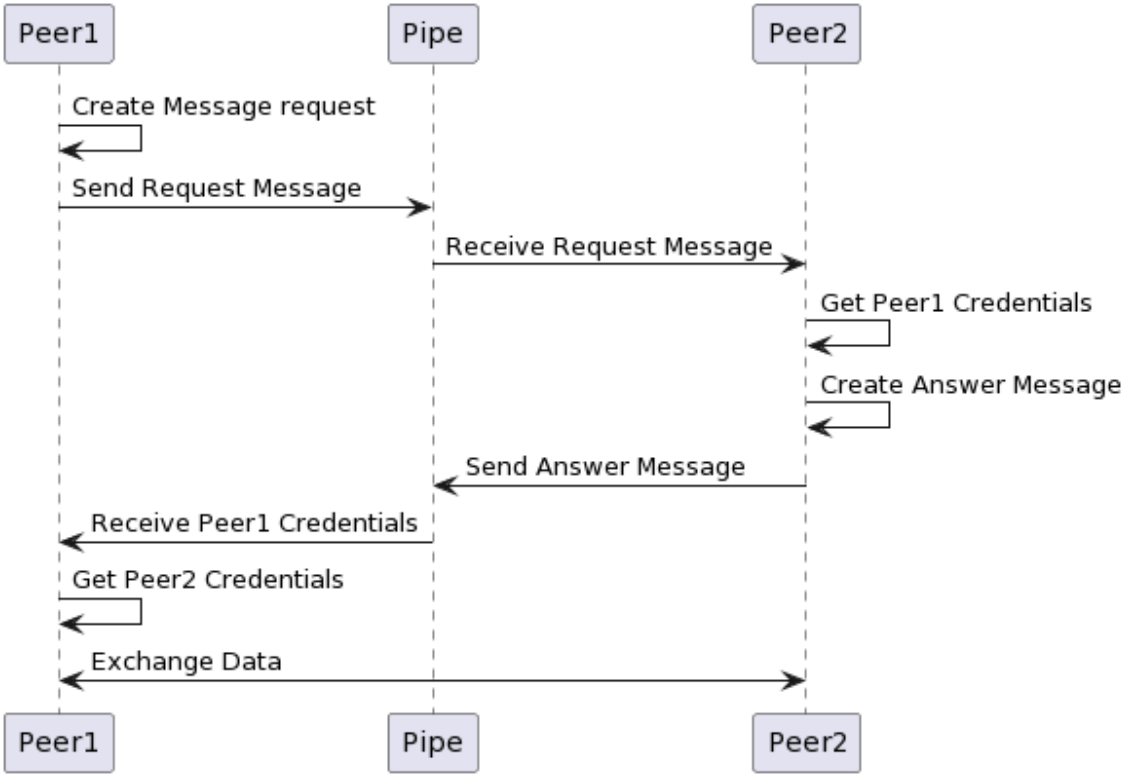


Figure 27. *The process of initiating an RTC call*

The Pipe can be any Protocol of data transfer, even manual messaging works, but in our case we are using a service dedicated to this task in our server, this makes it expansive to spin a connection from scratch every time which is why our implementation adds some heavy memoization mechanism to cache and retrieve connection peers when ever possible.

III.2 Server Architecture

Depending on the requirement of the project we concluded that it needs 3 Types of server services, each for a set of tasks.

- Authentication Server : This Service will take care of Authentication, whether by invoking local servers or by invoking the blockchain;
- Signaling Server : This Service will take care of sending short quick messages, that aren't time critical, i.e : bootstrap a video call, send a text message;
- Networking Server : This Service will take care of synchronizing the scene state across users, it is a real-time time service hence it is time critical;
- Storage Server : This Service will serve as the storage entry point for the project;

any other functionality needed can be built on top of these 4 services.

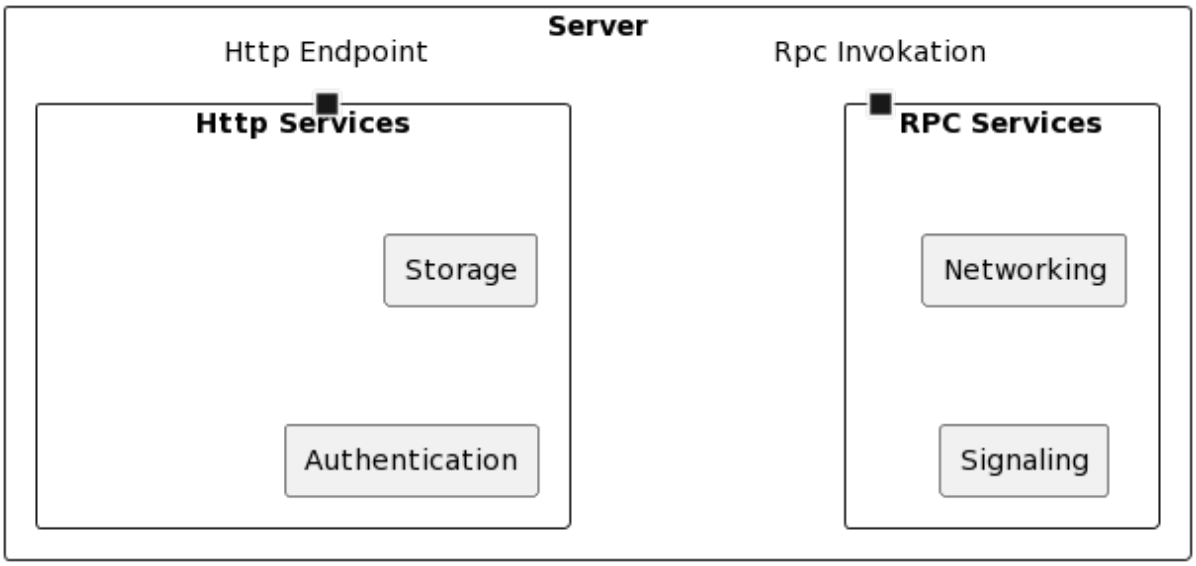


Figure 28. *An Overview of the server architecture*

The figure shows two types (2 types) of Services, Remote Procedure Call (RPC) services and Hypertext Transfer Protocol (HTTP) services, we differentiate between them by how often they spin into action, Networking and signaling services work very often inside the space, so

they rely on websockets to have an always on connection, while Storage and Authentication (Auth) services rarely get invoked hence why they are based on Http.

a) Http Services

Http services are comprised of both storage and Authentication services, these two services are characterized by the fact that they run rarely in a session.

i. Storage Service The Storage service start acting only when users are requesting or sharing documents, it's use fairly limited, it is basically a Create Read Update and Delete (CRUD) service for serialized Media files.

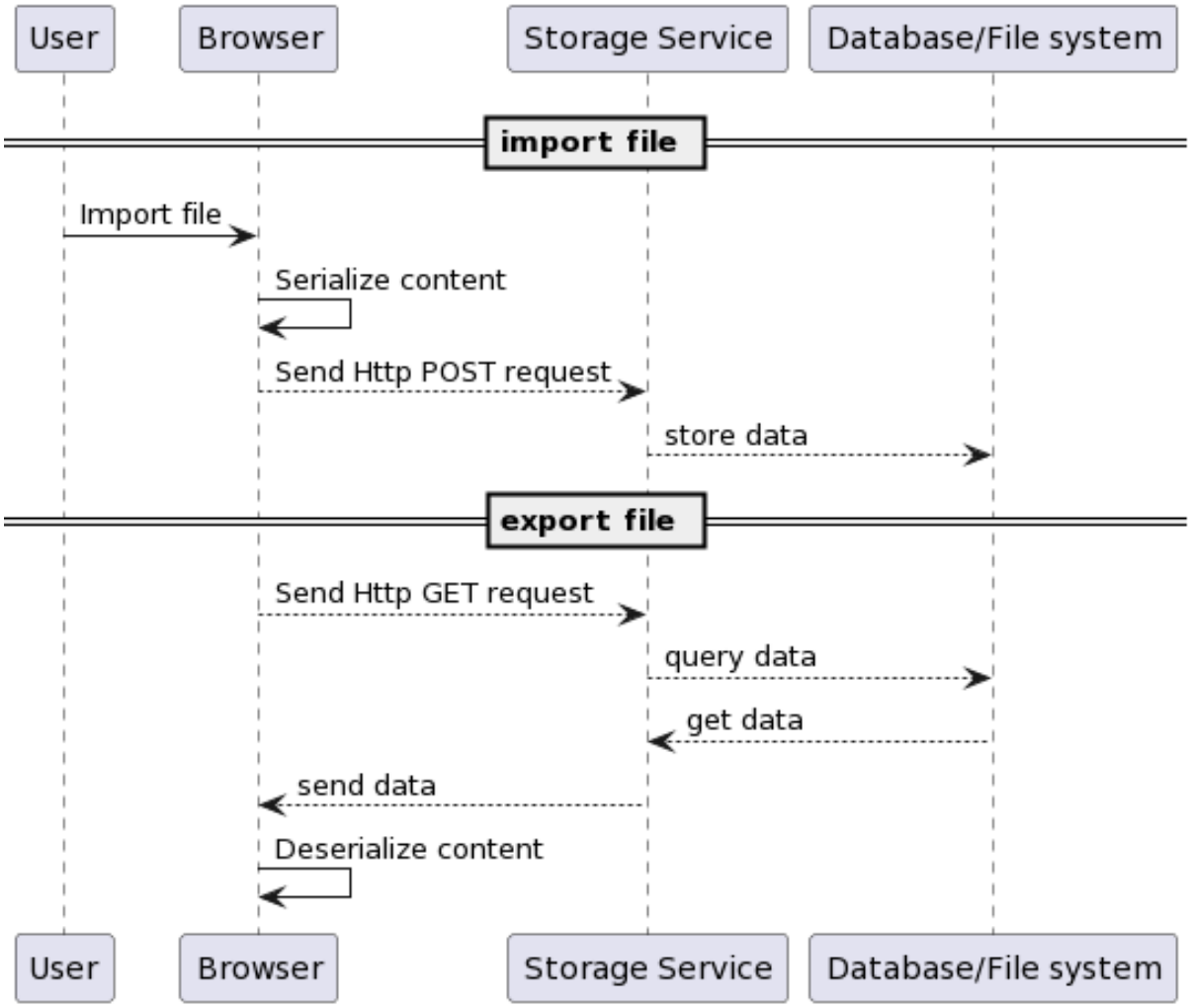


Figure 29. *An Overview of the process of file storage*

ii. Authentication Service The authentication as the name suggests is the service taking care of user authentication, it is a typical register/login service with extra feature inherited from the web3 nature of the project, in addition to using email and password, the service also uses a decentralized identity system, this latter relies on user wallets and smart contracts to store and retrieve user data. mostly those contracts are in the form of NFT tokens, these latter serve like passport stamps, and are used to identify the user.

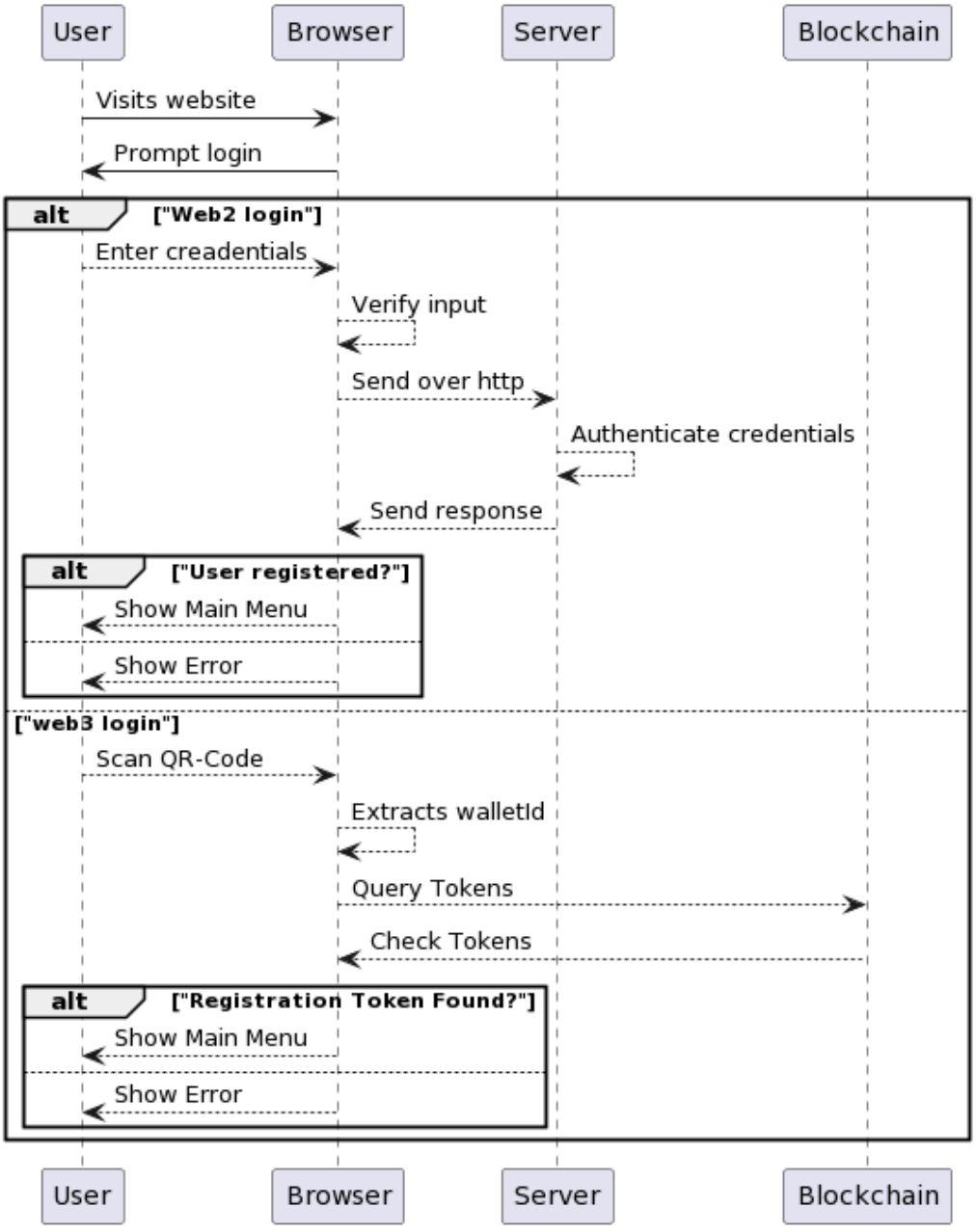


Figure 30. *An Overview of the process of authentication*

b) RPC Services

Rpc services are characterized by the fact that they are always spinning in a session, they are used to communicate with the server and with the blockchain, they rely on web sockets under the hood, and they use named functions tagged as RPCs to dispatch the requests.

i. Signaling Service Although signaling is a task that doesn't require an always on connections, it is a task that is often used in the system, that is why it is an RPC service, spinning an HTTP connection for each and every message that is sent is expansive.
One use case of this service as said before is bootstrapping a WebRTC Connection, another one would be synchronizing animation across network, that process is illustrated in the following diagram :

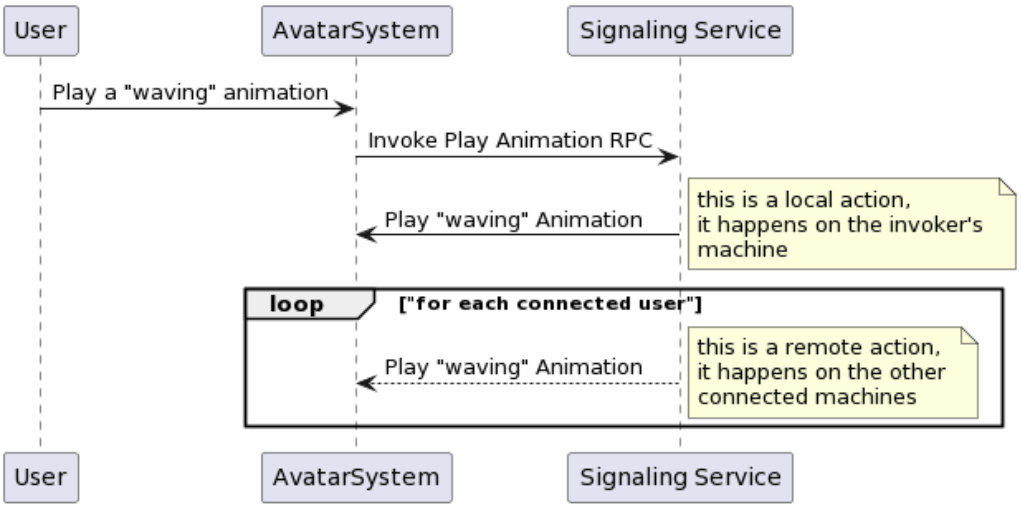


Figure 31. *An Overview of the process of signaling*

ii. Networking Service Networking service is the mechanism by which our system ensures that the scene is always in sync, it's used to synchronize state and actions across network, but its main concern is to allow users to explore collectively the spaces they join, and to allow users to interact with each other, and since the state of the scene changes every frame, this service needs to be efficient, fast and allow for real-time updates of the state on the fly. One use case of this service as stated before is synchronizing positions of users inside of a space,

the flow of the process goes as follow :

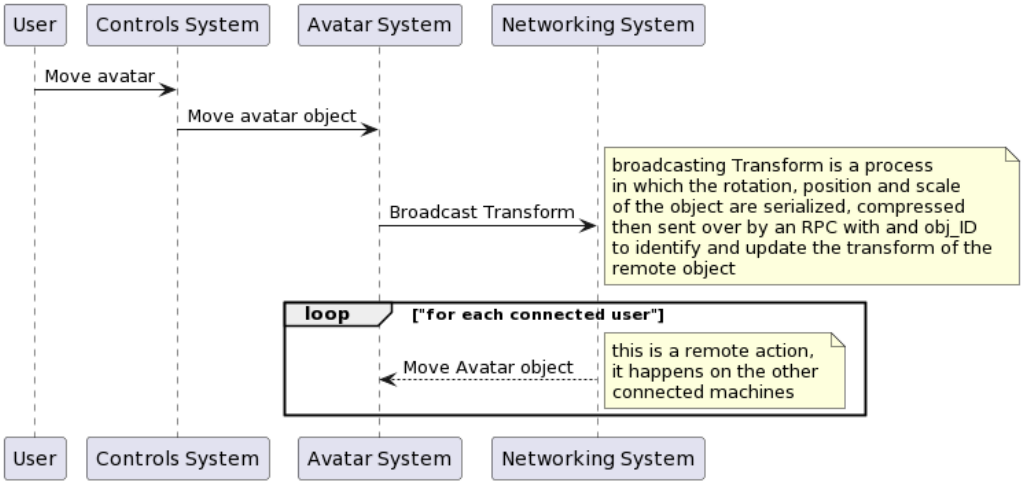


Figure 32. *An Overview of the process of transform synchronization*

IV Bundling

Bundling is the process of hooking these systems and libraries together in order for them to function, for the internal systems, the simple application of ECS suffices, avatar entities get their own subsystems, i.e : The controls system and Avatars System, and then the orchestrator object (the main manager of the scenes) gets the worker systems.
Some problems arise from the fact of how non trivial webgl builds interact with the browser own architecture, since our program will be compiled in phases

- 1st Phase : C# to Microsoft Intermidiate Language (MSIL) or coomonly known as Intermidiate Language (IL), this phase it the typical step in VM based runtime compilations;
- 2nd Phase : IL to CPP (C++), this is a phase introduced by the Unity engine to compile IL code to low level C++ code that is ready to be compiled for browser envirements;
- 3rd Phase : C++ to Web Assembly (WASM), this phase is crucial for the build to run on the browsers, it uses WASM as a compilation target using emscripten as a c++ compiler;

Now due to the fact that Wasm is a new web standard access to many of the browsers APIs is limited or in some cases non-existant, which proved to be problematic when trying to do voice based communications, audio streaming is not supported yet by unity because of the mentioned limitations of WASM, the solution was to do it entirely in a host website and use Unity / Js interop to handle the processes, which added a new layer of indirection but gave us control over the communication layer the lowest level possible.

So in actuality the webRTC call bootstrapping happens as follows : The figure 33 shows that in order to start an RTC call the interop layer is used to remedy the fact that we cant handle the calls from within the build itself.

Secondly when it comes to file sharing, some problems arise due to what have been state before, WASM limitations, but this time the interop layer too suffers from a limitation Unity by itself doesn't have a file picker for WebGL builds, so interop with JS was necessary to use the browser's native file picker, until now it's all sound and clear, but by the nature of how Javascript interacts with WASM the file buffers that can be sent back and forth are limited in size, and Media files which we desire to import are of sizes exceeding that limit (the limit is around 1.6mb), the solution was to use an intermediate file storage and pass only addresses around and fetch them when needed, of course doing this will be problematic for the server since it will be handling a very big amount of file transfers each session, the Concept mechanism of the file transfer then was adapted to incorporate the InterPlanetary File System (IPFS) as a storage facility [**6**], and use its protocols to fetch those documents (See figure 34).

V Conclusion

To conclude, during the implementation of the project, we have added many features, and functionalities, solved problems as we go, as we've seen the project is an interconnected web of services and systems, interoperating, but done in a way standalone system adheres to the spec defined, the main goal of this phase was to develop the Proof Of Concept and prove it is doable, and later refactoring and industrializing may be added to the project.

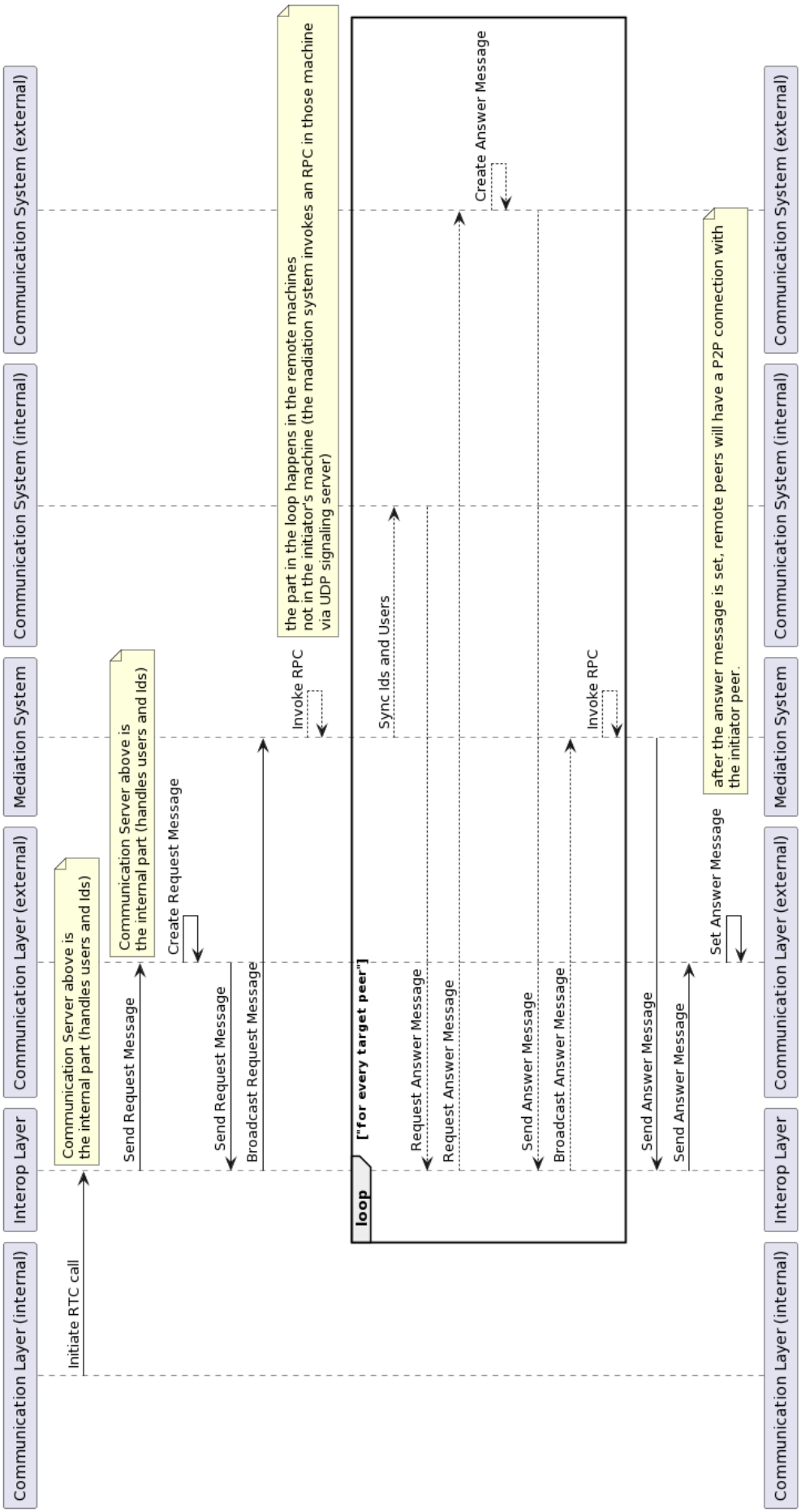


Figure 33. *An in depth look at RTC call bootstrapping*

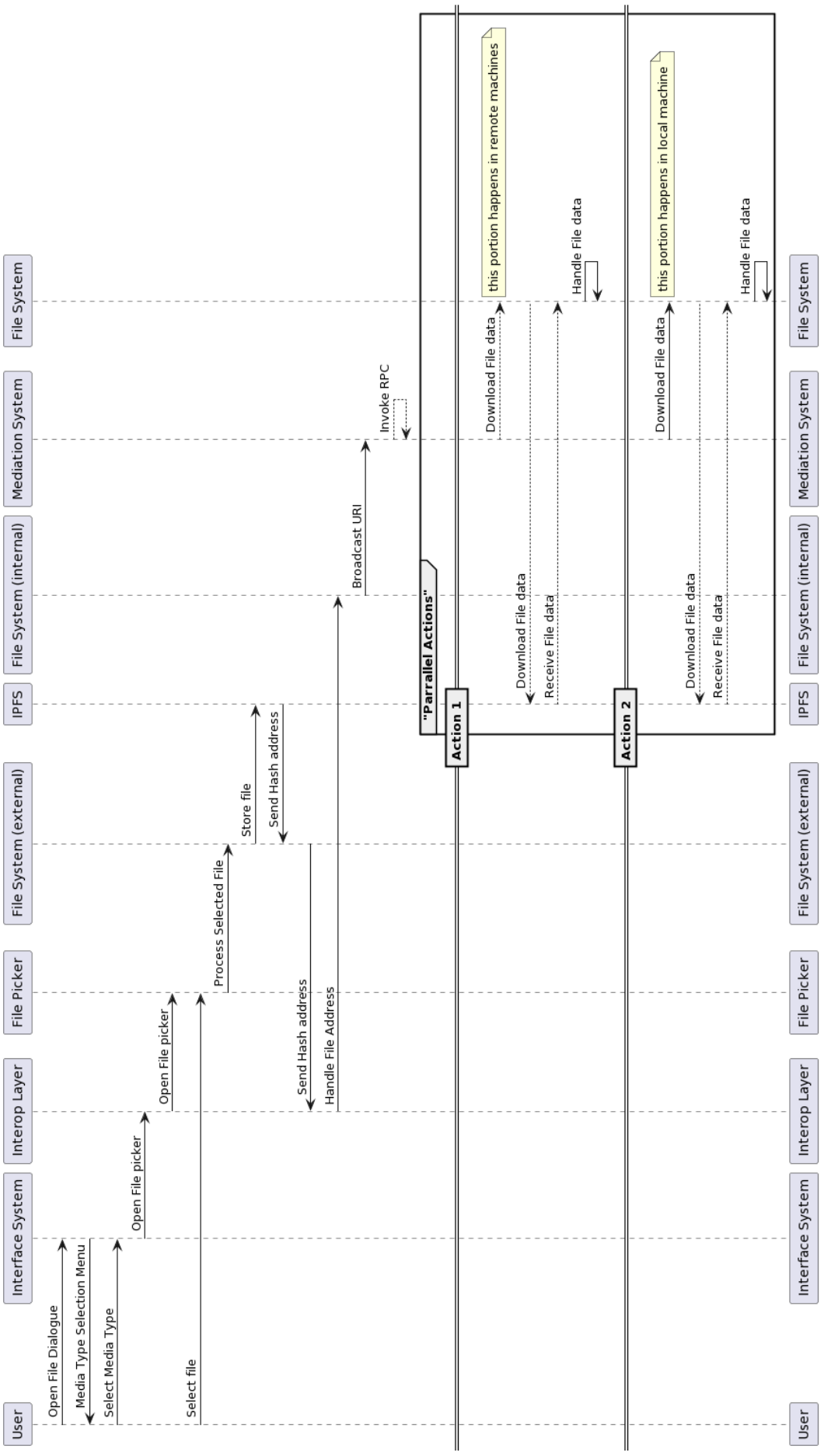


Figure 34. *An in depth look at File import process*

Chapter 4

Programming

This chapter will delve into the implementation process and tools used to achieve that, as well as the results of the implementation, and the various tests that were performed.

First we will see the tooling utilized to implement the software, which among those we cite : unity the game engine, photon the networking engine, and .Net as the backend technology ... etc, those are some of the technologies used in the internship.

In the next part we will showcase the results of the implementation, both the front-end and the backend, and explain some example functionalities.

and lastly we will show and explain various tests done to verify the soundness and correctness of the software.

I Introduction

Every software life-cycle passes from the implementation phase, and this phase is one of the most important phases of the project's life, to do it, the Development team uses various technologies, and tools to bring the project to life, this chapter will go through some of these tools.

II Development environment

In this Section we will explore the various tools and technologies used during the implementation of the project, we can separate them into different categories, tooling, organization, technologies.

- Tooling : the various tools such us IDEs, text editors and other software that were used to directly implement the project;
- Organization : the tools that were used to coordinate and organize the team's tasks and work;
- Technologies : the various SDKs and APIs ... etc that were used to implement certain features in the project;

II.1 Tooling

Tooling software as stated above is the collection of application used to host the implementation process of the project.

- Visual Studio (VS) Code : a versatile code editor developed by Microsoft (MSFT), extended by the C# extensions and intellisence it provides a powerful coding and debugging context;

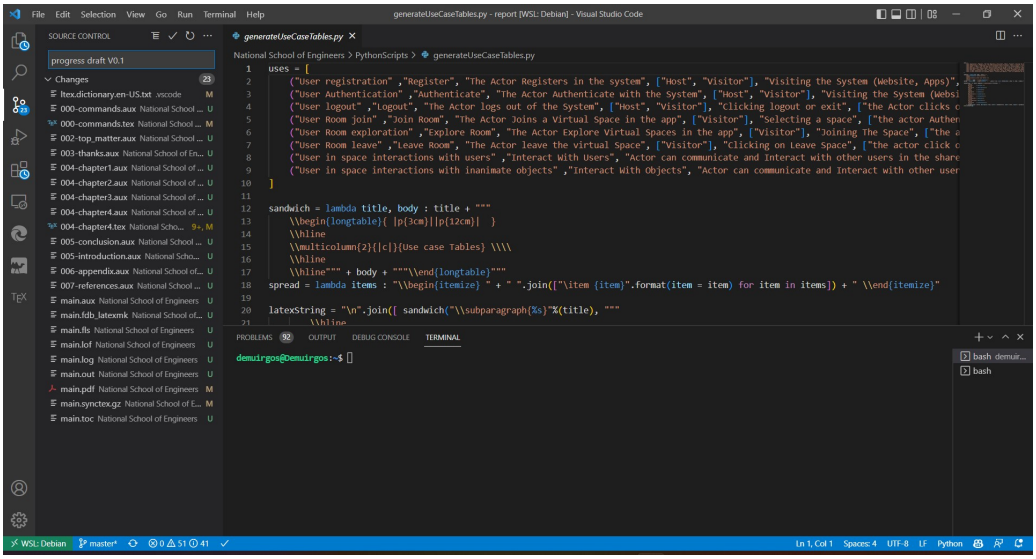


Figure 35. *Screenshot of Visual studio code*

- Unity : a game engine used for rendering 3D visuals, it provides a light weight runtime to build games and 3D software on top of, it uses C# as scripting Language,
  one of the main aspects of this engine is its ability to target multiple build targets mainly : webgl, windows, android, macos, ios ... [**2**].

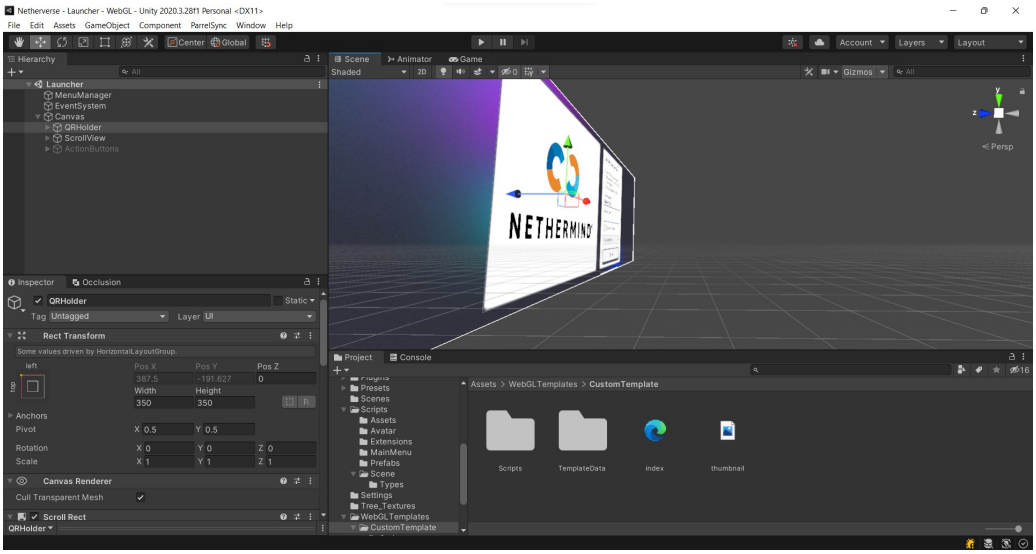


Figure 36. *Screenshot of Unity Editor*

- Edge : a browser developed on top of chromium used to debug WASM and WebGL builds.

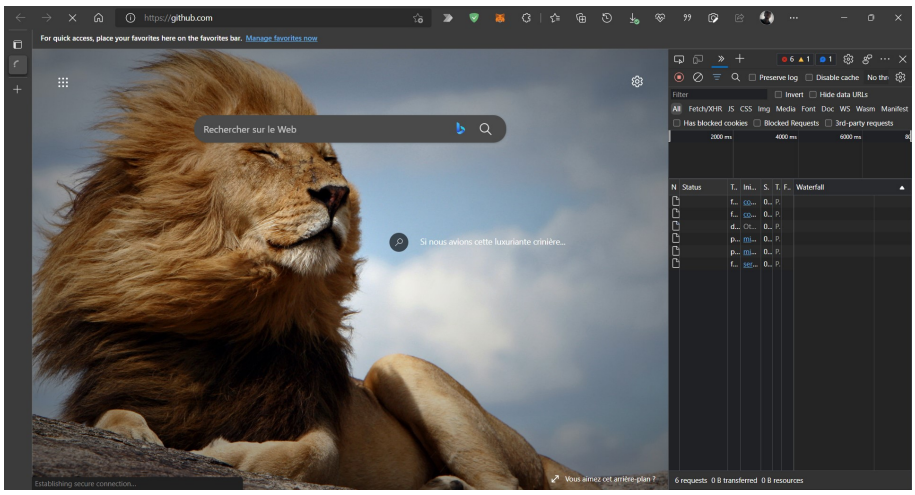


Figure 37. *Screenshot of Microsoft Edge*

II.2 Organization

For organization we used some unconventional for team coordination and some conventional software to coordinate the implementation process. From these we mention :

- Slack :Slack is a messaging program designed specifically for the workplace. it offers persistent chat rooms organized by topic, private groups, and direct messaging, we use it to talk and communicate with the team;

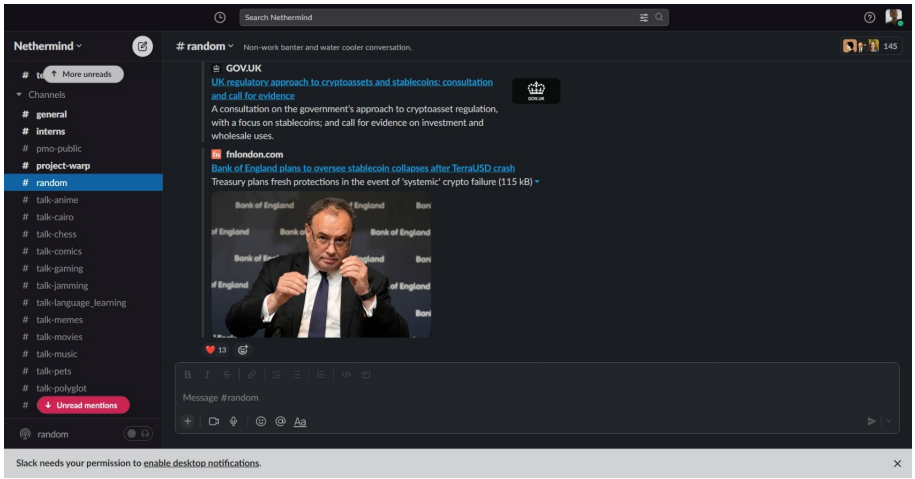


Figure 38. *Screenshot of Slack App*

- Notion :Notion is a project management and note-taking software. Notion is software designed to help members of a company or organization coordinate deadlines, objectives, and assignments for the sake of efficiency and productivity, and we use it in the company to share memos, file, and other documents, as well as manage tasks;

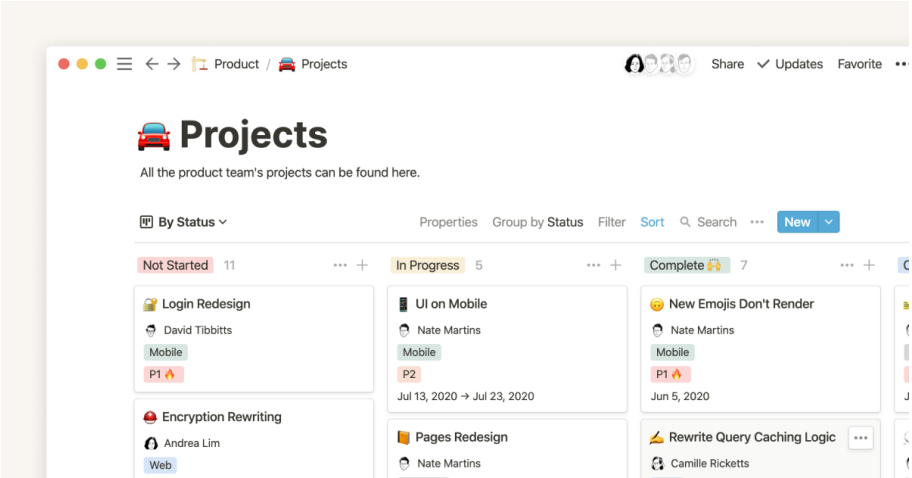


Figure 39. *Screenshot of Notion App*

- Github : GitHub, Inc. is a provider of Internet hosting for software development and version control using Git. It offers the distributed version control and source code management functionality of Git, and we use it to coordinate coding, and deploy Tests as CI/CD process;

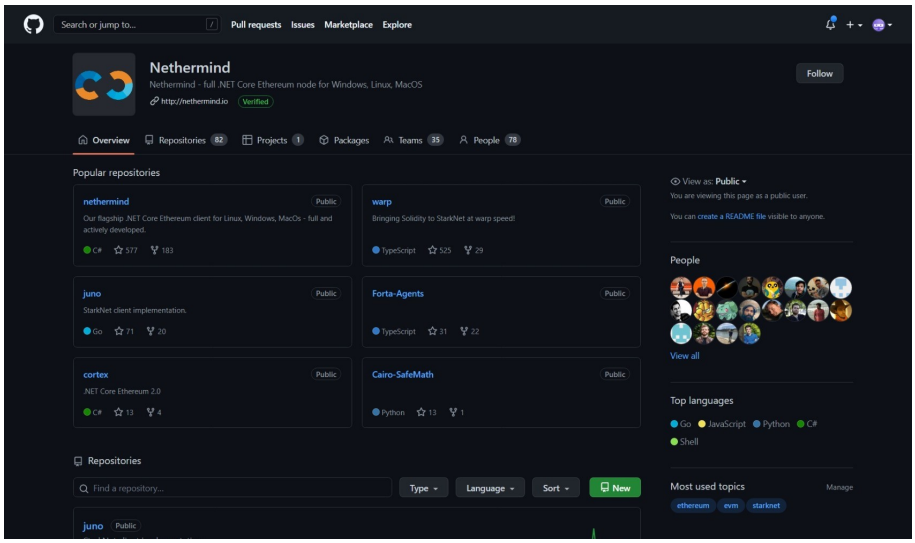


Figure 40. *Screenshot of Github website*

II.3 Technologies

When it comes to technologies our team used various technologies to achieve its goal, these tools can be divided into different categories, such as : programming languages, Software Development Kit (SDK), API and Frameworks.

- Programming Languages : we used C# and Typescript as main programming languages, and solidity as a smart contract language.
  - C# for its robustness, since it was imposed by the Unity engine there wasn't much of a choice there, many aspects that made C# was it having a good type system, a very fast runtime and overall a good ecosystem;
  - Typescript, we used it as an alternative to Javascript, since we wanted to benefit from Typescript's compiler to catch as much preventable bugs as possible, and syntax wise it is similar to C# which makes the switch seamless and not a big deal, we used it to implement the host website as well as the interop layer;
  - Solidity, its use cases were niche, as it is not a general purpose programming language as the others it was only used rarely to develop some custom smart contracts for the blockchain interaction;
- SDK : we used Unity's Photon as a networking engine, and Unity's Moralis SDK as a blockchain interop SDK.
  - Photon : This is a networking solution built on top of unity, it provides many basic functionalities to bootstrap multi-player game development [**4**];
  - Ready Player Me (RPM): This is a toolbox that allows for dynamic generation of avatar at pre-runtime to be used inside 3D spaces [**5**];
  - Moralis: This is an SDK that allows for interacting with wallets and the blockchain, it provides ease of use and simplicity and a good integration with Unity[**7**];

- Frameworks :
  - IPFS : The InterPlanetary File System is a protocol and peer-to-peer network for storing and sharing data in a distributed file system. IPFS uses content-addressing to uniquely identify each file in a global namespace connecting all computing devices, and we use it to share documents when possible, or save state for presistense;
  - .Net : For the backend we used .Net (Dotnet 6), as it provides a very rich ecosystem, and it uses C# as well, allowing us to share most type definition, and have one coherent homogeneous codebase or at least to some degree [**3**];

III Application

The application is for the main part a website that hosts a webgl build, other build targets exist such as android and windows but they aren't prioritized at the moment, the process goes as follows :
The user visits the applications's website :

III.1 Avatar Selection

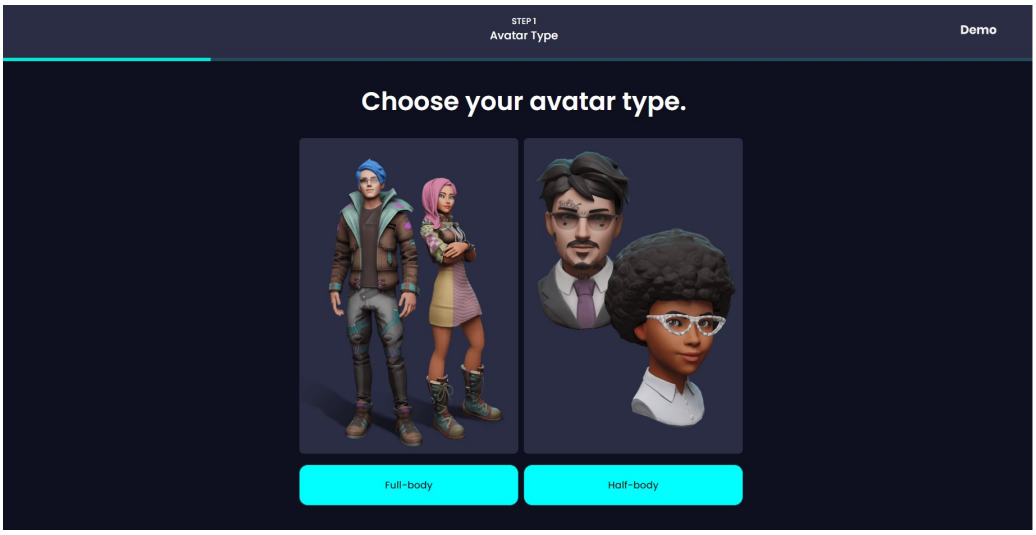


Figure 41. *First Menu in the website*

the image above shows the first menu in the Avatar Selection process, VR avatars mostly use bodyless Mannequins, but we also full body rigged avatar mannequins, actually they are the main mode of use in the Netherverse.

Next we can either login to the RPM service to fetch our avatars if we have already made some, or we can choose a gender to proceed to create a custom one on the fly.

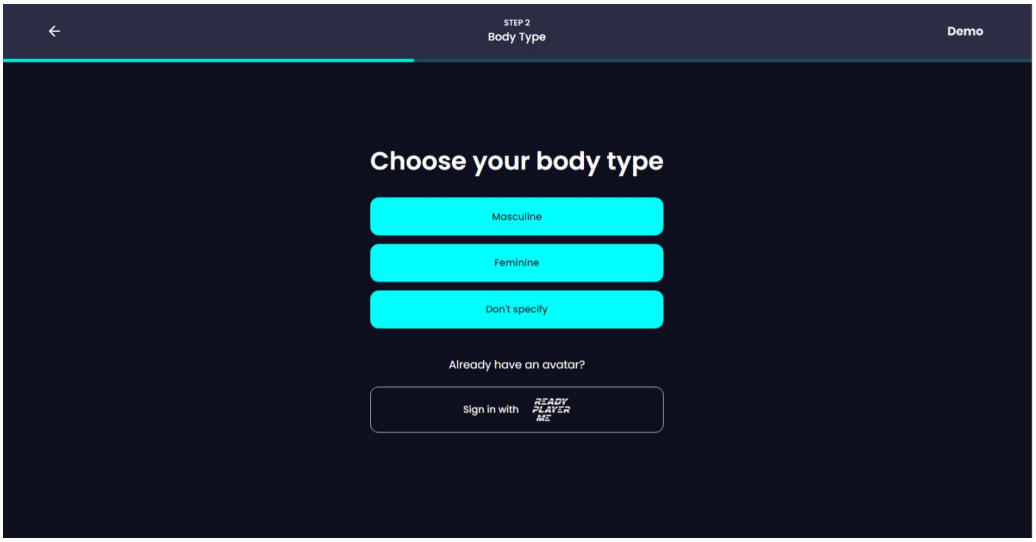


Figure 42. *Gender Selection in the website*

after selecting the Avatar type, we then either take a photo or select a photo from the gallery, so the application can approximate an avatar dynamically on the fly, or we can pick an premade avatar from the list as shown bellow

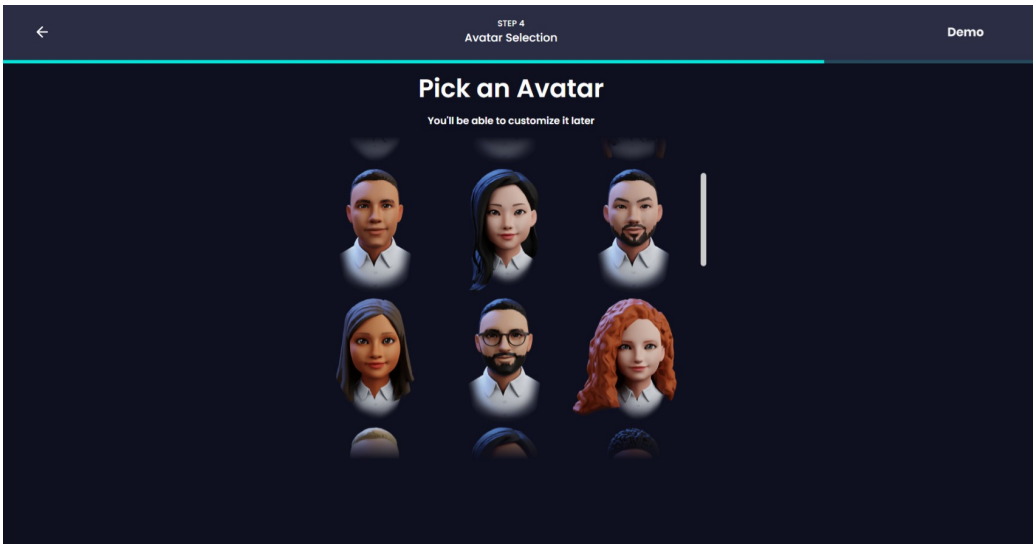


Figure 43. *Avatar Selection Menu in the website*

when we pick our avatar, then we get to customize certain aspect of it's appearance, such as the color, the skin, the clothes, the hair style ... etc, the image bellow shows the UI of the customization process.

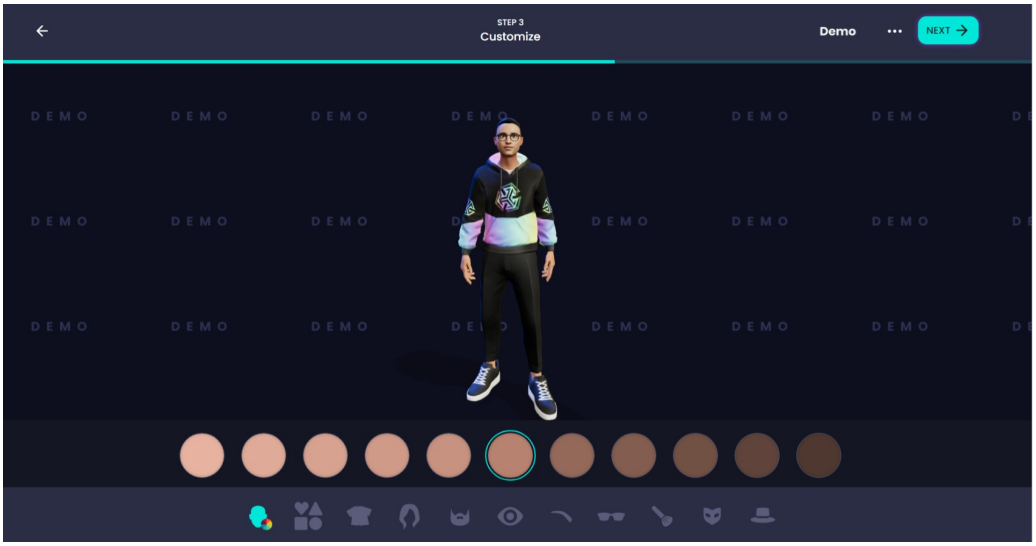


Figure 44. *Avatar customization in the website*

the changes can be done using the toolbox in the bottom and the avatar will update dynamically to show the changes.

Figure 45. *Avatar customization after changing some aspect of the avatar*

III.2 Spaces Selection

After finishing the Avatar selection and customization, we close the RPM menu and we can start the Netherverse, but before we can join, we need to fill out a few details, such as the name, the space we want to join ... etc. the next UI depends on some external factors such as whether the user has a web3 wallet extension in the browser, if they don't they will see the classical login page :

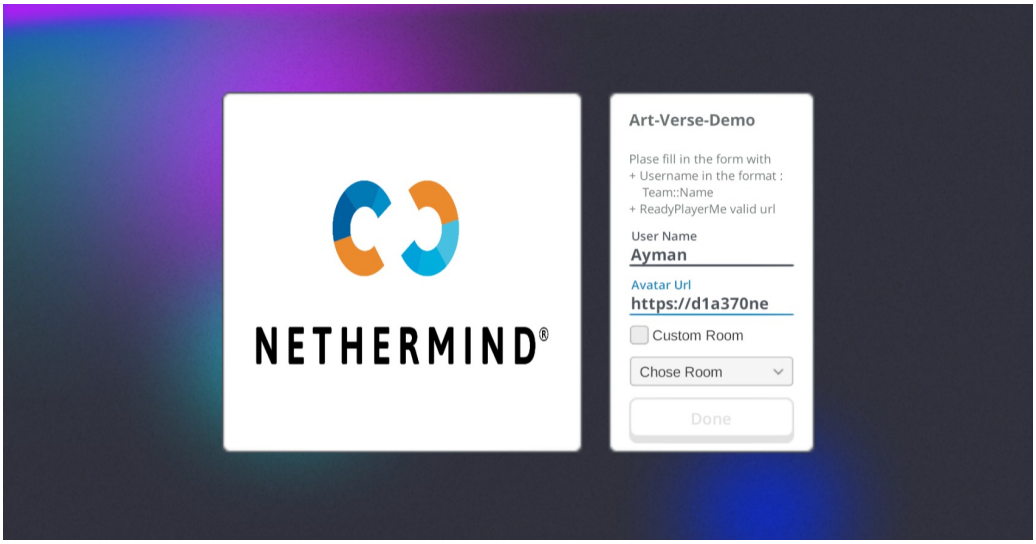


Figure 46. *Login page in the website using normal Login*

and if they actually do have a wallet they will be shown this UI instead :

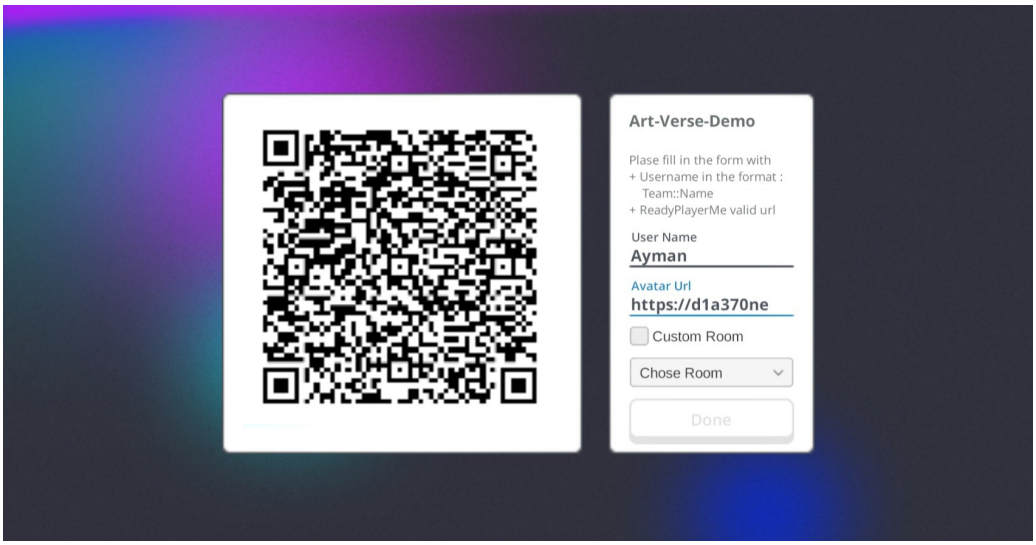


Figure 47. *Login page in the website using web3 Login*

after filling those forms, we can join the Netherverse, and we can start exploring the spaces.

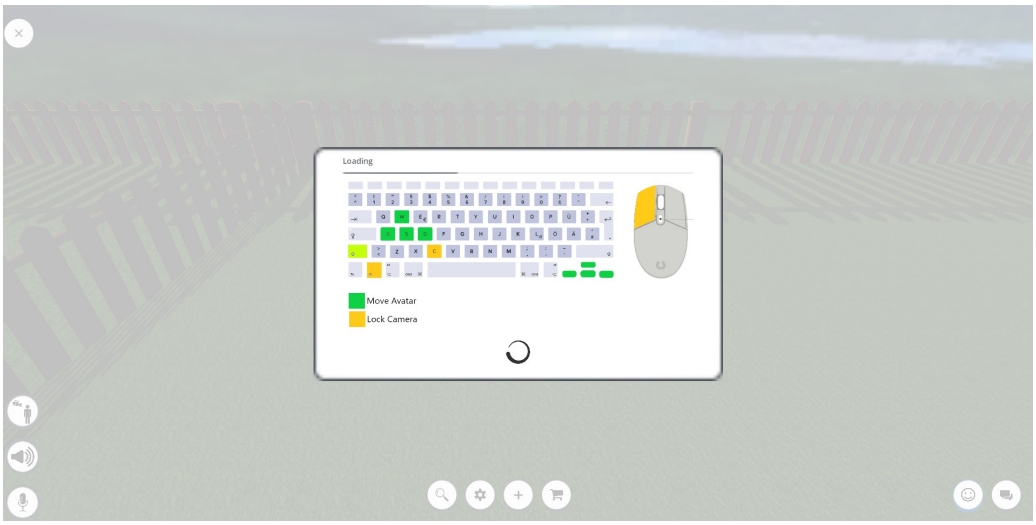


Figure 48. *Netherverse Loading screen, it shows the controls, while downloading the Avatar Model*

III.3 Spaces Exploration

Once loading is finished we can start moving around and interacting with the world, the image bellow shows the UI of the Netherverse.

Figure 49. *Netherverse UI*

we can do various things inside the Netherverse such as import custom objects, and chat and talk with other people.

Figure 50. *Importing a Budhist priest Model in the space*

the image above shows the user importing a Status Model inside the space using the import Menu, other objects can be imported as well such as : Videos, Images, Text, and Audio.

Figure 51. *using the chat UI to send a local message*

III.4 Netherverse in Action

And lastly when other users join the same space as us, we can see their avatars and chat with them.

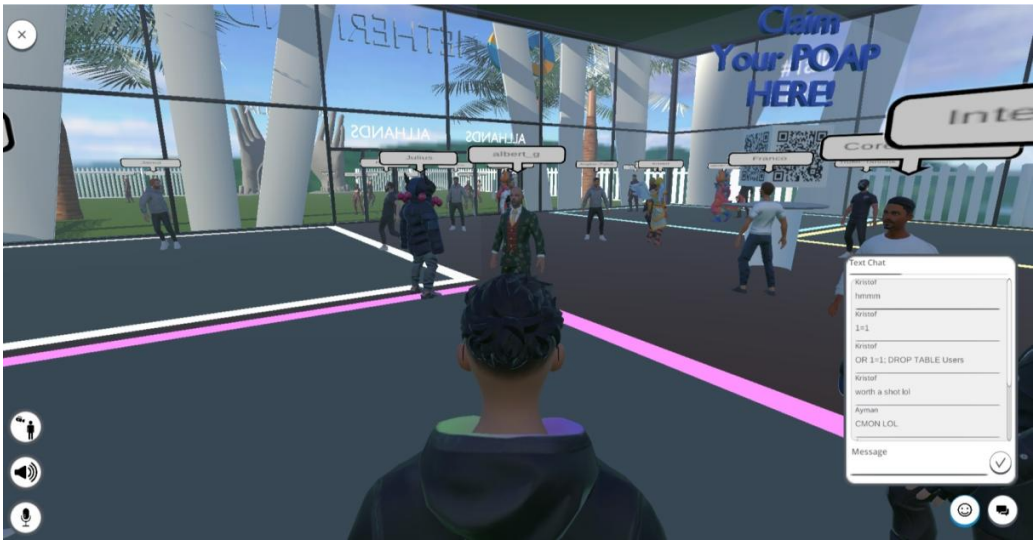


Figure 52. *chatting inside local zones with other people*

Figure 53. *exploring the space with other users*

The images above show a view of the Netherverse during one of its global testing moments, when a milestone was achieved, having more than 40+ people inside concurrently, talking and exploring the space.

The other side of the project is Arts, the Netherverse spaces are customizable to fit the needs of the host, as an example if the host want an art gallery they can make it inside of the netherverse.

Figure 54. *Netherverse Art Gallery*

The image above shows a Netherverse art gallery with a few models inside and some Images, and the user can interact with them as well as other users in that space.

IV Tests

When it comes to the test we did one global test, at the AllHands event (an internal event), it was designed to stress test, the system to see how much it can handle the stress test result were :

- Client Side limits : the test proved that the hardware of the client poses a limitation on the software itself, which put in a situation where we needed to chose what target Specifications we are going to target, or are we going to impose limits on certain aspects to keep the project resource friendly;
- Server Side limits : the test also proved that the server architecture has limits when it comes to broadcasting, not technical limits but financial ones, as every message costs money, and we can calculate the price of each session by the following equation : nbrOfCCUs * msgPrice, (nbrOfCCUs : Number of Concurernt Users, msgPrice : Message Price), message price varies according to it's byte size, so to remedy this, we

moved away from json serialization toward binary serialization, and introduced a hard cap for message counts per room;

- Communication Side limits : Due to the nature by which we handled communication, sounds is flat and bound to zones, which proved to be annoying and noisy, to solve that we emulated 3D sound in the browser context and now sound is bound to zones and a special distance field function;

The following is a chart depicting the result of the test where performance stands for the average between the reported clients Frames Per Second (FPS):

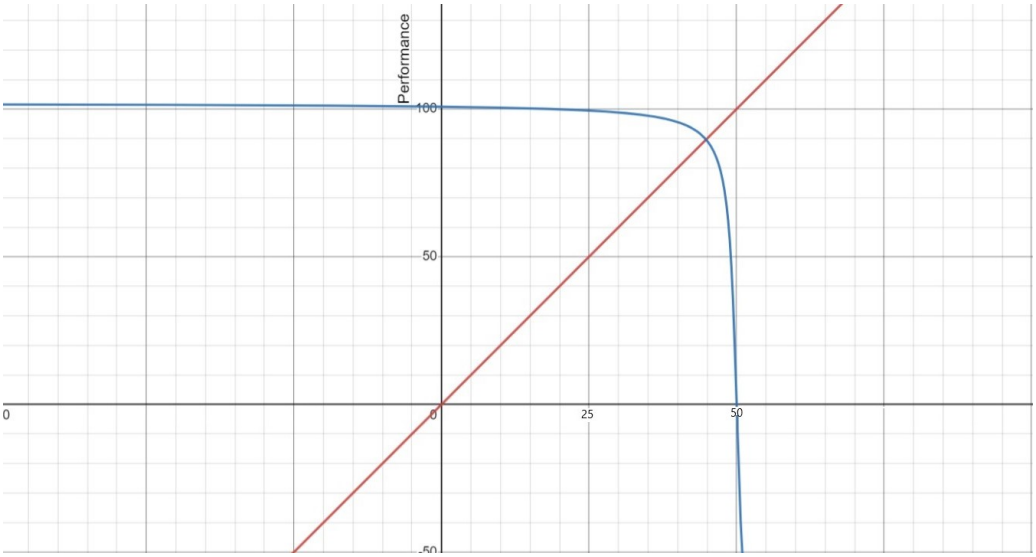


Figure 55. *the result of the benchmark*

The figure shows that performance starts degrading rapidly around 50 CCUs, mainly because of the client's hardware specs, so for now we added the ability to create dynamic room based on CCU count, and capped the size limit at 25, while migrating the networking layer to a new architecture that used space culling and a hybrid P2P/Server network pipes.

V Conclusion

In conclusion, this was a showcase of the project, we've seen avatar selection menu, login phase, and the 3D space showcase, the UI is a work in progress, but made simple to adhere with WEB3

style, and in the ”Netherverse in Action” we saw the 3D space having many users interacting with each other. we couldn’t showcase the rest of the project as it is all implicit systems working in the background, but having the build hosting at least 40 people is a proof of them functioning.

Conclusion

Socializing with each other is one of the main aspect of the human conditions, in the amount of interaction we can have is limitless, but across the web it's another story, it is rigid, and it's not always easy to communicate with others.

Our project comes to mediate the best of both worlds, allow for rich interaction and communication, and allow for the exchange of knowledge and information. while not requiring physical presence, this gives flexibility to social events and interactions.

The starting point for the project was identifying the missing features in the already existing products in the market, then on top of that building a set of requirements and needs it needs to fulfill in order to be a viable product.

After specifying the functional requirements of the projects, we designed a global concept of the project, and we kept rectifying it adding more details until we had the big picture on which we build a detailed model of the project.

The last chapter was dedicated to present the various tools used to implement the project, and the methods followed to achieve the goals of the project. This project's impact was of the utmost importance to the parties involved, and most importantly on us interns, since it allowed us to explore various team work methods, and to learn numerous technologies, and to be able to use them to build a better product.

In retrospect there are parts of the software that could've been done better, such as bootstrapping voice chat or networking, but overall the project was a success, and it was a great experience.

Webography

[1] WebRTC Documentation,
https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API

[2] Unity Documentation, https://docs.unity3d.com/Manual/index.html

[3] Dotnet Documentation, https://docs.microsoft.com/en-us/dotnet/

[4] Photon Documentation,
https://documentation.help/Photon-Unity-Networking-1.91/general.html

[5] ReadyPlayerMe Integration Docs,
https://docs.readyplayer.me/ready-player-me/integration-guides/unity

[6] How the IPFS works, https://docs.ipfs.io/concepts/how-ipfs-works/

[7] Moralis documentation, https://docs.moralis.io/introduction/readme